

# PartialLoading: User Scheduling and Bandwidth Allocation for Parameter-sharing Edge Inference

Guanqiao Qu, *Graduate Student Member, IEEE*, Qian Chen, *Member, IEEE*,  
Xianhao Chen, *Member, IEEE*, Kaibin Huang, *Fellow, IEEE*, Yuguang Fang, *Fellow, IEEE*

**Abstract**—By provisioning inference offloading services, edge inference drives the rapid growth of AI applications at the network edge. However, achieving high task throughput with *stringent latency requirements* remains a significant challenge. To address this issue, we develop a parameter-sharing AI model loading (PartialLoading) framework for multi-user edge inference, which exploits two key insights: 1) the majority of latency arises from loading AI models into server GPU memory, and 2) different AI models can share a significant number of parameters, for which redundant loading should be avoided. Towards this end, we formulate a joint multi-user scheduling and spectrum bandwidth allocation problem to maximize task throughput by exploiting shared parameter blocks across models. The intuition is to judiciously *schedule* user requests to reuse the shared parameter blocks between consecutively loaded models, thereby reducing model loading time substantially. To facilitate solution finding, we decouple the problem into two sub-problems, i.e., user scheduling and bandwidth allocation, showing that solving them sequentially is equivalent to solving the original problem. Due to the NP-hardness of the problem, we first study an important special case called the “bottom-layer-sharing” case, where AI models share some bottom layers within clusters, and design a dynamic programming-based algorithm to obtain the *optimal* solution in polynomial time. For the general case, where shared parameter blocks appear at arbitrary positions within AI models, we propose a greedy heuristic to obtain the sub-optimal solution efficiently. Simulation results demonstrate that the proposed framework significantly improves task throughput under deadline constraints compared with user scheduling without exploiting parameter sharing.

**Index Terms**—Edge AI, edge inference, edge computing, 6G, user scheduling, bandwidth allocation, task throughput maximization.

## I. INTRODUCTION

In the realm of Artificial Intelligence (AI), next-generation wireless networks will become the key infrastructure to facilitate big data processing and intelligence extraction on an enormous scale [1]–[5]. Reflecting this progression, IMT-2030 has highlighted “Artificial Intelligence and Communication” as a pivotal usage scenario of the sixth-generation (6G) mobile networks to support ubiquitous AI applications [6]–[9]. By equipping the network edge with localized AI computing capabilities, 6G edge intelligence considerably reduces

data transmission latency compared with cloud AI, thereby revolutionizing various critical sectors, such as healthcare, transportation, industrial automation, and education [10]–[17].

Despite the advantages of low transmission latency, edge computing possesses much less powerful computing capabilities than its cloud counterpart [18]–[23]. This limitation can hinder its response time, especially during peak hours. For instance, Augmented Reality (AR) applications have stringent end-to-end (E2E) latency requirements of 7-15 ms [24], making it highly challenging to support such latency-sensitive requests simultaneously from a large number of users. To address this, various resource allocation schemes have been developed, particularly on the joint optimization of communications and computing resources, to accommodate bursty user requests [25], [26]. However, the performance is inherently constrained by the availability of computing resources, such as limited GPU resources on an edge server. From a computing perspective, system efficiency can be further enhanced by batching user requests to perform inference computation in parallel, thereby reducing inference latency [27]–[29]. Nevertheless, while inference batching parallelizes multiple requests for the same AI model, it does not reduce the latency during model loading/swapping, which often dominates the total inference time. As shown in Fig. 1, loading an AI model into GPU memory accounts for an average of 88.94% of the total latency, whereas inference computing for a batch takes only a small fraction. Moreover, for large models with billions of parameters, e.g., GPT-2, model loading time becomes more significant, reaching 96.3% of the time [30]. This phenomenon underscores the critical need to develop edge inference systems with reduced model loading time.

To overcome the aforementioned limitations and accelerate the edge inference process, the key idea of this paper is to leverage *shared* parameters across models to eliminate redundant model loading. Specifically, for numerous AI models, such as convolutional neural networks (CNNs) or large language models (LLMs), downstream models can share a substantial portion of parameters, because a *usual practice* is to derive them from a pre-trained model based on layer freezing or parameter-efficient fine-tuning techniques [33]–[39], instead of training from scratch. As shown in Fig. 2, we pre-train a ResNet-50 model [31] on the CIFAR100 dataset [32] and then fine-tune it on two sub-datasets of CIFAR10 [32] with the bottom-layer freezing technique [37], creating two downstream models. As such, the bottom layers of the downstream models for Task 1 and Task 2 are shared from the pre-trained model. Importantly, the inference accuracy shows *no degradation*

Guanqiao Qu, Qian Chen, Xianhao Chen, and Kaibin Huang are with the Department of Electrical and Electronic Engineering, University of Hong Kong, Hong Kong SAR, China. (e-mail: gqqu@eee.hku.hk; qchen@eee.hku.hk; xchen@eee.hku.hk; huangkb@eee.hku.hk). Yuguang Fang is with the Department of Computer Science, City University of Hong Kong, Hong Kong SAR, China. (e-mail: my.fang@cityu.edu.hk). The work was supported in part by the Research Grants Council of Hong Kong under Grant 27213824. (Corresponding author: Xianhao Chen.)

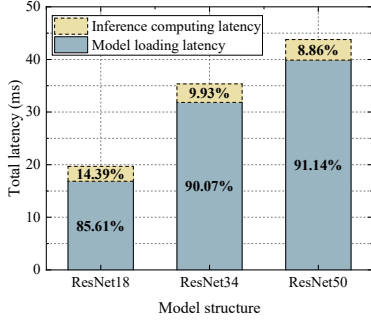


Fig. 1. Total latency across various model structures. Model loading refers to the process of loading an AI model to GPU memory, while inference computing includes data tensorization and batching, moving data to GPU memory, and forward propagation. The models are from the ResNet family [31], with a batch size of 32, evaluated on the CIFAR10 dataset [32]. Inference is executed on a Linux server equipped with an Intel Core i9-13900K CPU, a GeForce RTX 4090 GPU with 24 GB GPU memory, a Toshiba 8 TB SATA3 HDD, and two Kingston 32 GB DDR5 RAM modules.

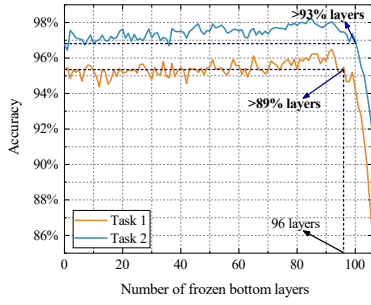


Fig. 2. Inference accuracy v.s. the number of frozen bottom layers in fine-tuned ResNet-50 models. Tasks 1 and 2, respectively, correspond to the classification of labels 0-4 and 5-9 in CIFAR10. This example shows different downstream models can share a significant proportion of layers without performance degradation compared with full-parameter fine-tuning (0 frozen layer).

compared with full-parameter fine-tuning (0 frozen layer), even up to 93% and 89% of bottom layers are shared, respectively. In fact, since parameter freezing can overcome overfitting, significant parameter sharing across models can be achieved with no accuracy loss or even with accuracy improvement, as demonstrated in many seminal works [37], [38], [40].

Motivated by the above observations, in this paper, we address the joint multi-user scheduling and spectrum bandwidth allocation problem for parameter-sharing edge inference to maximize task throughput (i.e., the number of served tasks) under latency and resource constraints. Specifically, we propose a parameter-sharing AI model loading (PartialLoading) framework by strategically scheduling and batching user inference requests while effectively allocating bandwidth to the scheduled users in each batch. Our optimization framework aims to schedule users to optimally reuse shared parameter blocks between consecutively loaded AI models, thereby saving model loading time. It is important to note that, while model loading has been widely recognized as a major bottleneck in computing systems, the existing groundbreaking solutions, such as in-memory computing [41], resort to proposing brand-new computing architecture, which faces incompatibility with existing commercial systems. Instead, this paper takes a pure *resource management* approach by only designing a novel

*scheduler* for wireless edge systems. Our main contributions are summarized as follows.

- 1) We propose a framework leveraging parameter sharing across AI models to enable partial model loading for edge servers. This framework addresses a joint user scheduling and spectrum bandwidth allocation problem in multi-user parameter-sharing edge inference systems to maximize the task throughput under latency, communication, and computing resource constraints.
- 2) To facilitate solution finding, we effectively decouple the problem into two sub-problems, i.e., user scheduling and spectrum bandwidth allocation, demonstrating that solving the original problem is equivalent to solving these two sub-problems sequentially. For the bandwidth allocation sub-problem, we derive the optimal closed-form expression for bandwidth allocation under given user scheduling solutions. Next, we show that the user scheduling sub-problem is an NP-hard problem and provide the necessary conditions for optimal user scheduling solutions.
- 3) Due to the NP-hardness of the problem, we first investigate an important special case of the original problem, where AI models within clusters share bottom layers. For this bottom-layer-sharing case, we first obtain the optimal model loading order, based on which a dynamic programming (DP)-based algorithm is proposed to obtain the optimal solution with polynomial-time complexity.
- 4) For the general case, where shared parameter blocks appear at arbitrary positions across models, we design a greedy heuristic algorithm to find a sub-optimal solution. Both of our algorithms considerably outperform the traditional user scheduling strategy without exploiting the parameter-sharing property of model loading.

The rest of this paper is organized as follows. Section II reviews the related work. Section III introduces the system model and the PartialLoading framework. Section IV formulates the task throughput maximization problem, presents the equivalent problem decoupling, derives the optimal bandwidth allocation policy, and establishes the necessary conditions for optimal user scheduling. Section V investigates a special case of the problem, while Section VI addresses the general case. Section VII presents simulation results. Finally, Section VIII concludes the paper.

## II. RELATED WORK

In multi-user edge inference, enhancing task throughput under latency constraints is a challenging issue. This difficulty primarily comes from the conflict between stringent task latency requirements and limited communication and computing resources on an edge server. To improve task throughput, a common approach is to leverage task batching, i.e., scheduling the requests for the same AI model into one inference batch [27]–[29], [42]–[46]. By batching them into an AI (sub)model, an edge server processes inference computing in parallel based on the parallel computing of GPU. Building on this approach, some research works optimize data selection for inference batching. However, these works focus

on scheduling users into a single inference batch [27], [42], [44]–[46], limiting their applicability in the real world. While some studies have also addressed the user scheduling problems for multiple batches, they make the simplified assumption that users request the same AI model [28], [29], [43]. In practice, an edge server typically hosts diverse AI models for its subscribers for different applications.

As alluded to earlier, loading AI models into the server memory is a major bottleneck in the inference process. To address this, Ogden et al. [47] design a model eviction policy that optimizes the models cached in memory for new inference requests by minimizing the cache miss penalty, where the penalty occurs when a requested model is not cached. However, it does not leverage parameter sharing across AI models on loading time. In [40], Padmanabhan et al. propose a loading strategy for edge inference with parameter-sharing models, where the next model is scheduled to maximize the overlapping of shared layers with the previous one, thereby reducing model loading time. Nevertheless, this heuristic fails to account for the data uploading and computing time and does not provide a provable performance guarantee. Moreover, these works overlook the communication constraints of edge servers, limiting the practicality of their approaches.

The key idea of our proposed framework is strategically scheduling user requests to reuse the shared parameter blocks between consecutively loaded AI models. However, while the performance gain is clear, solving the optimization problem is highly challenging. Specifically, assuming AI models are independent or there is only one AI model, the scheduling problems in edge inference can be typically mapped to knapsack problems and solved with standard DP algorithms [27], [29]. Unfortunately, when considering parameter sharing across models, the interdependencies among models significantly complicate user scheduling, causing standard DP procedures to fail. Moreover, incorporating bandwidth allocation further increases the complexity of the problem. To the best of our knowledge, this is the first paper to address the user scheduling problem in multi-user edge inference by exploiting parameter sharing across models.

### III. FRAMEWORK

#### A. Network Scenario

As illustrated in Fig. 3, we consider a single-edge multi-user scenario in wireless edge networks. A set of users,  $\mathcal{K} = \{1, 2, \dots, K\}$ , are associated with a wireless edge server (e.g., a server colocated with a base station). Each user uploads raw data or intermediate features to the edge server for processing, and then the server returns the inference results to users. Without loss of generality, we assume each user generates one task requesting a specific AI model, and each model can be requested by multiple users. The set of models is denoted as  $\mathcal{I} = \{1, 2, \dots, I\}$ , which are hosted on the edge server. We assume that all tasks arrive at time 0, and the time horizon is divided into  $T$  consecutive time slots with equal duration  $\Delta\tau$ , indexed by  $\tau \in \mathcal{T} = \{1, 2, \dots\}$ .

As a remark, unlike prior works that predominantly focus on an oversimplified *homogeneous* task modeling where all users

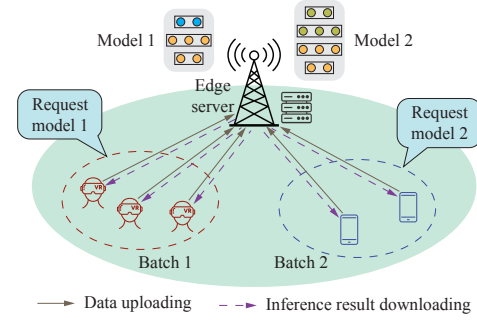


Fig. 3. Our considered single-edge multi-user scenario, where parameter blocks can be shared among AI models hosted on the server. In this figure, the first two layers (in orange) are shared between model 1 and model 2.

request the same AI model for inference [27], [29], this paper addresses a more practical *heterogeneous* task modeling by considering multiple AI models/tasks hosted on an edge server, which is prerequisite for applying our scheduling policy.

#### B. Parameter-sharing AI Model Library

We consider the parameter-sharing in the AI model library  $\mathcal{I}$ . The set  $\mathcal{J} = \{1, 2, \dots, J\}$  represents the parameter blocks of the models in  $\mathcal{I}$ . Given that AI models may contain billions of parameters, a parameter block refers to a set of model parameters, such as a (sub-)layer, a model block, or even a backbone model, to reduce the complexity of the problem. This applies to a broad range of AI models like CNNs, Transformers, and LLMs. The definition of a shared parameter block is formally presented below.

**Definition 1. Shared parameter block:** A parameter block is referred to as a shared parameter block if it is contained in more than one AI model within a considered set of models.

Moreover, when a parameter block refers to a layer, a shared parameter block is also called a *shared layer* in this paper.

#### C. Inference Batching with Partial Model Loading

To enhance task throughput, the server processes tasks in batches. Specifically, multiple tasks requesting the same model can be batched together. The batch sequence is denoted by  $n \in \mathcal{N} = \{1, 2, \dots, N\}$ , where the edge server processes batch sequentially in ascending order of  $n$ . In batch  $n$ , only users requesting the same AI model can be scheduled. Let a binary variable  $x_{n,k}$  be the decision variable for user scheduling, with  $x_{n,k} = 1$  indicating that the server processes user  $k$ 's task in batch  $n$ . As illustrated in Fig. 4, each inference batch consists of the following three steps.

(1) **Data uploading:** Scheduled users upload raw data or features to the edge server. The uploaded data is cached in the system memory of the edge server after reception.

(2) **Partial model loading:** Once all data from the scheduled users is uploaded, the edge server loads the specified AI model from the disk to system memory and then to GPU memory. To reduce latency, the shared parameter blocks between two consecutively loaded models in the current and the previous batch are reused without reloading.

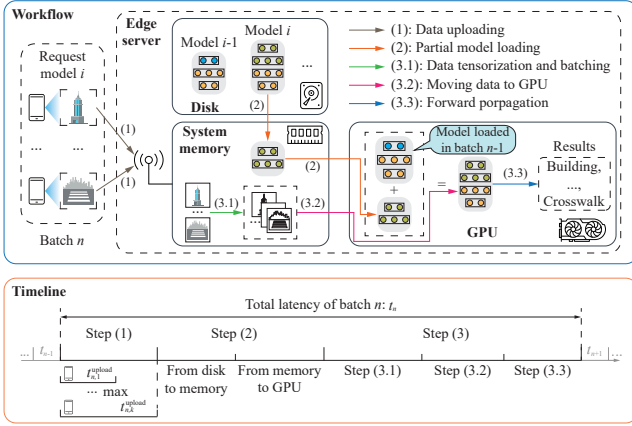


Fig. 4. Workflow and timeline of an inference batch. We use object classification/detection applications as an illustrative example. Mobile devices offload the captured images to the edge server and request model  $i$  in batch  $n$ . Since model  $i$  shares the first two layers (in orange) with model  $i-1$ , which has been loaded in the previous batch (batch  $n-1$ ), the edge server only loads the last two layers (in green) of model  $i$  into the GPU memory for inference. The main goal of this paper is to schedule users into a sequence of batches to optimally leverage the parameters shared across models, thereby enhancing task throughput under latency constraints.

(3) **Inference computing:** Inference computing comprises three steps. (3.1) *Data tensorization and batching*: The edge server converts the uploaded raw data into tensors, which can be skipped if users upload preprocessed features directly. Then, the edge server assembles the tensorized data into a single batch. (3.2) *Moving data to GPU*: The edge server moves the batched data from the system memory to the GPU memory for further processing. (3.3) *Forward propagation*: The edge server feeds the batched data into the AI model in the GPU memory, performs forward propagation, produces inference results, and sends the results back to users.

Due to the small data size of the inference results and the high transmit power of the edge server, the time of downloading inference results is ignored [28]. As a result, the total latency for processing batch  $n$  is expressed as

$$t_n = t_n^{\text{up}} + t_n^{\text{load}} + t_n^{\text{comp}}, \quad (1)$$

where  $t_n^{\text{up}}$  represents the data uploading time of scheduled users,  $t_n^{\text{load}}$  denotes the time for partial model loading, and  $t_n^{\text{comp}}$  is the inference computing time. The detailed timeline of an inference batch is illustrated in Fig. 4, and further details will be provided in the following subsections.

User  $k$  can be scheduled and served in batch  $n$  if it can obtain the inference result within the E2E latency requirement, satisfying

$$x_{n,k} \sum_{n'=1}^n t_{n'} \leq T\Delta\tau, \quad \forall n \in \mathcal{N}, \quad \forall k \in \mathcal{K}, \quad (2)$$

where  $\bar{T} = T\Delta\tau$  denotes the E2E latency requirement. Additionally, since user  $k$  can be scheduled in at most one batch, it follows that

$$\sum_{n \in \mathcal{N}} x_{n,k} \leq 1, \quad \forall k \in \mathcal{K}. \quad (3)$$

#### D. Uplink Communication Time

We consider an FDMA uplink channel between the users and the edge server. In each batch, the total bandwidth  $B$  of the edge server is divided into multiple sub-channels, each allocated to a single user. The bandwidth allocated to user  $k$  in batch  $n$  is denoted by

$$B_{n,k} = y_{n,k}B, \quad (4)$$

where  $y_{n,k} \in [0, 1]$  is the ratio of the allocated bandwidth, satisfying

$$\sum_{k \in \mathcal{K}} y_{n,k} \leq 1, \quad \forall n \in \mathcal{N}. \quad (5)$$

Moreover, the expected data uploading rate between user  $k$  and the edge server is given by  $\bar{C}_{n,k} = B_{n,k}\bar{R}_k$ , where  $\bar{R}_k = \log_2 \left( 1 + \frac{P_k d_k^\alpha}{N_0} \right)$ ,  $P_k$  is the spectral density of the transmit power of user  $k$ ,  $d_k$  is the distance between user  $k$  and the server,  $\alpha$  is the path loss factor, and  $N_0$  is the spectral density of the additive white Gaussian noise. Thus, the data uploading time of user  $k$  in batch  $n$  is

$$t_{n,k}^{\text{up}} = \begin{cases} \frac{x_{n,k} D_k}{y_{n,k} B \bar{R}_k}, & \text{if } x_{n,k} = 1, \\ 0, & \text{if } x_{n,k} = 0, \end{cases} \quad (6)$$

where  $D_k$  is the size of the data uploaded by user  $k$ . Furthermore, since the partial model loading step begins only after all scheduled users in batch  $n$  finish uploading their data,  $t_n^{\text{up}}$  in (1) is expressed as

$$t_n^{\text{up}} = \max_{k \in \mathcal{K}} \{t_{n,k}^{\text{up}}\}. \quad (7)$$

#### E. Partial Model Loading Time

In the partial model loading step of batch  $n$ , an edge server only loads the parameter blocks not shared between models in batches  $n-1$  and  $n$ . Therefore,  $t_n^{\text{load}}$  in (1) is

$$t_n^{\text{load}} = V_1(\bar{S}_n) + V_2(\bar{S}_n), \quad (8)$$

where  $\bar{S}_n = \sum_{j \in \mathcal{J}} S_j (\rho_{n,j} - \rho_{n-1,j})$  is the size of parameter blocks to be loaded in batch  $n$ , and  $S_j$  is the size of parameter block  $j$ . Here,  $\rho_{n,j} = 1 - \prod_{i \in \mathcal{I}_j} \prod_{k \in \mathcal{K}_i} (1 - x_{n,k})$ , where  $\mathcal{I}_j$  is

the set of models containing parameter block  $j$ ,  $\mathcal{K}_i$  is the set of users requesting model  $i$ .  $\rho_{n,j} = 1$  indicates that parameter block  $j$  is included in the request model in batch  $n$ , where  $\rho_{0,j} = 0$ . Thus,  $\rho_{n,j} - \rho_{n-1,j} = 1$  implies that parameter block  $j$  has not been loaded in batch  $n-1$  and needs to be loaded in batch  $n$ . Moreover, functions  $V_1(\cdot)$  and  $V_2(\cdot)$  denote the loading time of data from the disk to system memory and from system memory to GPU memory, respectively, which are monotone increasing functions with data size.

**Remark 1.** This paper considers a commonly used architecture in Linux systems, comprising disks, system memory, and GPUs with the NVIDIA GPU architecture. In general,  $V_1(\cdot)$  depends on not only the bandwidth between the disk and system memory but also the page cache of the system memory [48], [49], whereas  $V_2(\cdot)$  can be approximated as the ratio of the data size to the bandwidth between the system memory and GPU memory [50].



### F. Inference Computing Time

The inference computing time increases linearly as the batch size grows [51]–[53]. Given  $x_{n,k}$ ,  $t_n^{\text{comp}}$  in (1) is

$$t_n^{\text{comp}} = \sum_{i \in \mathcal{I}} \left[ 1 - \prod_{k \in \mathcal{K}_i} (1 - x_{n,k}) \right] \left( \mu_i \sum_{k \in \mathcal{K}_i} x_{n,k} + \beta_i \right), \quad (9)$$

where  $1 - \prod_{k \in \mathcal{K}_i} (1 - x_{n,k}) = 1$  indicates model  $i$  is loaded in batch  $n$ ,  $\mu_i$  and  $\beta_i$  are constants related to the model structure and GPU performance when performing inference with model  $i$ , and  $\sum_{k \in \mathcal{K}_i} x_{n,k}$  is the batch size of batch  $n$  for model  $i$ .

## IV. TASK THROUGHPUT MAXIMIZATION PROBLEM

### A. Problem Formulation

Our framework aims to maximize task throughput by addressing the joint user scheduling and bandwidth allocation problem within the edge server's GPU memory capacity, communication constraints, and users' task latency requirements. The problem formulation is given as follows.

$$\mathcal{P1} : \max_{\mathbf{X}, \mathbf{Y}} U(\mathbf{X}) = \sum_{n \in \mathcal{N}} \sum_{k \in \mathcal{K}} x_{n,k} \quad (10a)$$

$$\text{s.t. (2), (3), (5),} \quad (10b)$$

$$\sum_{i \in \mathcal{I}} A_i \left( \sum_{k \in \mathcal{K}_i} x_{n,k} \right) \leq Q, \quad \forall n \in \mathcal{N}, \quad (10c)$$

$$\sum_{i \in \mathcal{I}} \left[ 1 - \prod_{k \in \mathcal{K}_i} (1 - x_{n,k}) \right] \leq 1, \quad \forall n \in \mathcal{N}, \quad (10d)$$

$$y_{n,k} \leq x_{n,k}, \quad \forall n \in \mathcal{N}, \quad \forall k \in \mathcal{K}, \quad (10e)$$

$$x_{n,k} \in \{0, 1\}, \quad \forall n \in \mathcal{N}, \quad \forall k \in \mathcal{K}, \quad (10f)$$

$$y_{n,k} \in [0, 1], \quad \forall n \in \mathcal{N}, \quad \forall k \in \mathcal{K}, \quad (10g)$$

where  $x_{n,k} \in \mathbf{X}$  and  $y_{n,k} \in \mathbf{Y}$  denote the decision variables for user scheduling and spectrum bandwidth allocation, respectively. In (10c),  $Q$  is the edge server's maximum available GPU memory, and function  $A_i \left( \sum_{k \in \mathcal{K}_i} x_{n,k} \right)$  is the peak GPU memory required for forward propagation with model  $i$  under batch size  $\sum_{k \in \mathcal{K}_i} x_{n,k}$ , which is a monotone increasing function [40]. (10d) ensures at most one model is loaded into GPU memory in batch  $n$ . (10e) guarantees user  $k$  is not allocated bandwidth unless it is scheduled in batch  $n$ .

### B. Equivalent Problem Decoupling

$\mathcal{P1}$  is a mixed-integer nonlinear programming problem coupling  $\mathbf{X}$  and  $\mathbf{Y}$ . To simplify the solution process, we derive the optimal closed-form expression for spectrum bandwidth allocation under  $\mathbf{X}$ . The details are as follows.

**Proposition 1.** *Given any user scheduling decision in batch  $n$ , the corresponding optimal spectrum bandwidth allocation*

*in batch  $n$  must minimize  $t_n^{\text{up}}$ . For given  $\mathbf{X}$ , the minimum value of  $t_n^{\text{up}}$  is*

$$t_{n,\min}^{\text{up}} = \sum_{k \in \mathcal{K}} \frac{x_{n,k} D_k}{B \bar{R}_k}, \quad (11)$$

*and the corresponding optimal spectrum bandwidth allocation under  $\mathbf{X}$  is*

$$y_{n,k} = \begin{cases} \frac{x_{n,k} D_k}{B \bar{R}_k \sum_{k' \in \mathcal{K}} \frac{x_{n,k'} D_{k'}}{B \bar{R}_{k'}}}, & \text{if } x_{n,k} = 1, \\ 0, & \text{if } x_{n,k} = 0. \end{cases} \quad (12)$$

*Proof.* The proof is provided in Appendix A.  $\square$

Building on Proposition 1, we can decouple  $\mathcal{P1}$  without compromising the optimality, as stated in the following theorem.

**Theorem 1.** *Solving the original problem  $\mathcal{P1}$  is equivalent to first solving the below sub-problem  $\mathcal{P2}$  and then determining the optimal spectrum bandwidth allocation using (12) based on the obtained user scheduling.*

$$\mathcal{P2} : \max_{\mathbf{X}} U(\mathbf{X}) \quad (13a)$$

$$\text{s.t. (3), (10c), (10d), (10f),} \quad (13b)$$

$$x_{n,k} \sum_{n'=1}^n \left( t_{n',\min}^{\text{up}} + t_{n'}^{\text{load}} + t_{n'}^{\text{comp}} \right) \leq T \Delta \tau, \quad \forall n \in \mathcal{N}, \quad \forall k \in \mathcal{K}. \quad (13c)$$

*Proof.* From Proposition 1, for any given  $\mathbf{X}$ , the optimal spectrum bandwidth allocation from (12) results in the minimum data uploading time in (11), which is independent of  $\mathbf{Y}$ . Therefore, after substituting (11) into  $\mathcal{P1}$  and eliminating the constraints on  $\mathbf{Y}$ , we obtain  $\mathcal{P2}$  that only depends on  $\mathbf{X}$ . Solving  $\mathcal{P2}$  obviously yields the optimal  $U(\mathbf{X})$ . Therefore, solving the user scheduling decision from  $\mathcal{P2}$ , followed by determining the spectrum bandwidth allocation from (12), preserves the optimality of the solution to  $\mathcal{P2}$ . This completes the proof.  $\square$

Thereafter, we focus on solving  $\mathcal{P2}$  in the rest of this paper.

### C. Problem Mapping

$\mathcal{P2}$  is a combinatorial optimization problem, as it involves selecting a user-scheduling solution across batches from a discrete set of feasible user-to-batch assignments under constraints. To facilitate analysis, we equivalently reformulate  $\mathcal{P2}$  into  $\mathcal{P3}$  with the objective of minimizing the number of unserved users as follows.

$$\mathcal{P3} : \min_{\mathbf{X}} K - U(\mathbf{X}) \quad (14a)$$

$$\text{s.t. (3), (10c), (10d), (10f), (13c).} \quad (14b)$$

$\mathcal{P3}$  is NP-hard since it can be mapped to a single machine batch scheduling problem to minimize the number of late jobs under latency constraints with a set-up time [54]. Specifically, in the mapped problem, selected jobs are grouped into batches

to be processed sequentially on a single machine, with a set-up time before processing at the beginning of each batch. Similarly, in  $\mathcal{P3}$ , scheduled users are batched and served by the edge server, with  $t_{n,\min}^{\text{up}} + t_n^{\text{load}}$  being the set-up time. In fact, even in a simplified case of  $\mathcal{P3}$ , where  $t_{n,\min}^{\text{up}} + t_n^{\text{load}}$  is constant and does not vary with  $n$ , the resultant problem has been proven to be NP-hard [55]. Therefore, the general form of  $\mathcal{P3}$  and  $\mathcal{P2}$  are NP-hard.

#### D. Necessary Conditions of the Optimal Solution to $\mathcal{P2}$

To narrow down the feasible user-scheduling solution space of  $\mathcal{P2}$ , we analyze the scheduling rules for users requesting the same model. The following theorem provides the necessary conditions for the optimal user scheduling  $\mathbf{X}^*$  in  $\mathcal{P2}$  within the batches of each model.

**Theorem 2.**  $\mathbf{X}^*$  must satisfy the following conditions.

- 2.1 Users requesting the same AI model must be scheduled in the same or consecutive batches. Moreover, for a given AI model  $i$ , users in  $\mathcal{K}_i$  must be scheduled in ascending order of  $p_k = \frac{D_k}{R_k}$ .
- 2.2 For consecutive batches requiring the same model, all batches except the last one should admit the maximum number of users within the GPU memory constraint.

*Proof.* The proofs of Theorems 2.1 and 2.2 are presented in Appendices B and C, respectively.  $\square$

**Remark 2.** Theorem 2.1 establishes a user scheduling rule that ensures users with the shortest uploading time given unit spectrum bandwidth allocation are scheduled first within the batches of any given AI model, thereby significantly simplifying the problem.

For ease of presentation, we reorder the indices of users in  $\mathcal{K}_i$  in ascending order of  $p_k$  in subsequent sections.

#### V. SPECIAL CASE: CLUSTERED AI MODELS WITH BOTTOM LAYER SHARING

Although we have derived the necessary conditions for the optimal user scheduling within batches of a single model,  $\mathcal{P2}$  remains challenging to solve. Considering parameter sharing across models, user scheduling across batches determines the model loaded in each batch and further affects the model loading time for the next batch. Thus,  $t_n^{\text{load}}$  in  $\mathcal{P2}$  varies across batches and depends on model loading order, rendering conventional approaches for combinatorial optimization problems impractical. Specifically, with this dependency, standard DP algorithms fall short due to the need for exhaustive searches over all feasible model loading orders. Additionally, although branch-and-bound algorithms can produce the optimal solution, their worst-case time complexity is exponential with the size of the model library. In this section, to develop a polynomial-time algorithm for real-time solution finding, we focus on an important special case called the “bottom-layer-sharing” case, where the optimal user scheduling strategy can be obtained very efficiently.

We first introduce the special case. Recalling that a layer is called a shared layer if it appears in at least two models, we

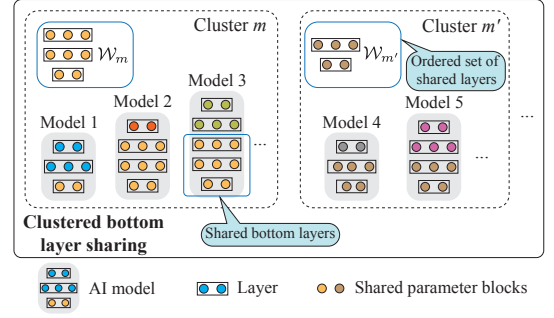


Fig. 5. An illustrative example of bottom layer sharing within two model clusters,  $m$  and  $m'$ . Within each model cluster, models share bottom layers. In cluster  $m$ , the first layer of model 1 and the first three layers of model 2 and 3 are shared bottom layers. Similarly, in cluster  $m'$ , the first two layers of models 4 and 5 are shared bottom layers.

formally define the special case in Definition 2 and provide an illustrative example in Fig. 5.

**Definition 2. Clustered bottom layer sharing:** Consider a set of AI models that can be partitioned into disjoint clusters  $\mathcal{M} = \{1, 2, \dots, M\}$ , with  $\mathcal{I}_m$  denoting the set of models in cluster  $m$ . Clustered bottom layer sharing occurs when each model  $i \in \mathcal{I}_m$  consists of two parts: (1) the first  $l_i$  shared bottom layers, corresponding to the first  $l_i$  layers in  $\mathcal{W}_m$ , where  $\mathcal{W}_m = \{1, \dots, W_m\}$  is an ordered set of shared layers in cluster  $m$ , with  $W_m = \max_{i \in \mathcal{I}_m} \{l_i\}$ , and (2) the remaining  $L_i - l_i$  layers, which are unique to model  $i$  and not shared with others, where  $L_i$  is the total number of layers in model  $i$ .

Note that different models in a cluster may have different numbers of shared bottom layers, and layer sharing across clusters is not considered, as shown in Fig. 5. For brevity, we use the *bottom layer sharing (BLS)* case to refer to the special case of clustered bottom layer sharing. The BLS case commonly exists in practice, as many AI models are fine-tuned from pre-trained models using bottom layer freezing techniques [37], [56], where one can freeze bottom layers in a pre-trained AI model while only updating the top layers towards downstream tasks. The rationale comes from the fact that bottom layers, say, in CNNs, typically extract low-level features (edges, corners, and basic shapes) that are reusable for various tasks. As such, freezing bottom layers in pre-trained models not only has minimal impact on the performance for new tasks but also prevents overfitting due to the smaller number of trainable parameters, as demonstrated in Fig. 2.

Beyond Theorem 2, we provide an additional necessary condition for the optimal user scheduling solution  $\tilde{\mathbf{X}}^*$  to  $\mathcal{P2}$  in the BLS case.

**Theorem 3.** In  $\tilde{\mathbf{X}}^*$ , users requesting models from the same cluster must be scheduled in the same or consecutive batches.

*Proof.* The proof is provided in Appendix D.  $\square$

With Theorem 3, we have the following corollary.

**Corollary 1. The optimal model loading strategy:** The edge server schedules the model loading order across clusters in

ascending order of cluster index  $m$ . For each cluster  $m$ , it schedules the model loading order for models in  $\mathcal{I}_m$  in ascending order of  $l_i$ , where  $i \in \mathcal{I}_m$ .

*Proof.* The proof is provided in Appendix E.  $\square$

**Remark 3.** Corollary 1 guarantees parameter blocks are never redundantly loaded in the BLS case. It enables us to determine the model loading order before scheduling users for each model, thus significantly simplifying the problem.

Thereafter, we reorder the models in  $\mathcal{I}_m$  by ascending order of  $l_i$  for ease of presentation. In the next subsection, the user scheduling strategy will be found with the above strategy.

#### A. The Solution Approach

With the optimal model loading order from Corollary 1, this subsection will develop a DP-based algorithm to compute the maximum task throughput and design a recursive algorithm to obtain the optimal user-scheduling decision  $\tilde{\mathbf{X}}^*$  in the BLS case. The details are as follows.

1) *Number of served users of a model:* Assuming that model  $i$  is loaded into the GPU memory after model  $\hat{i}$ , based on Theorem 2 and Corollary 1, we first determine the maximum number of users that can be served by model  $i$  in cluster  $m$  within  $\hat{\tau}$  time slots, which is denoted as

$$q_m(\hat{i}, i, \hat{\tau}) = \begin{cases} \max \left\{ \hat{k} \mid \Phi_m(\hat{i}, i, \hat{k}) \leq \hat{\tau} \Delta \tau \right\}, & \text{if } \hat{\tau} \in [1, T], i \in \mathcal{I}_m, \\ 0, & \text{if } \hat{i} = 0 \text{ or } \hat{\tau} = 0, \end{cases} \quad (15)$$

where  $\Phi_m(\hat{i}, i, \hat{k})$  is the total latency required to perform inference for  $\hat{k}$  users with model  $i$  loaded after model  $\hat{i}$  within model cluster  $m$ , and is given by

$$\begin{aligned} \Phi_m(\hat{i}, i, \hat{k}) &= \sum_{n=1}^{N'_i(\hat{k})} p_{z(n, i, \hat{k})} + V \left( \sum_{l=l_i+1}^{L_i} S'_{i,l} \right) \\ &+ \sum_{n=1}^{N'_i(\hat{k})} e(n, i, \hat{k}), \quad \hat{k} \in [0, |\mathcal{K}_i|], \hat{i} \in [0, i-1], \hat{i} \in \{0\} \cup \mathcal{I}_m. \end{aligned} \quad (16)$$

In (16), the first term  $\sum_{n=1}^{N'_i(\hat{k})} p_{z(n, i, \hat{k})}$  is the data uploading time for the first  $\hat{k}$  users in  $\mathcal{K}_i$  across multiple batches. Here,  $z(n, i, \hat{k}) = \min \{nb_{i, \max}, \hat{k}\}$  is the index of the user with the highest uploading time in batch  $n$ , where  $b_{i, \max}$  is the maximum allowed batch size for model  $i$ , determined by the GPU memory capacity and the peak memory required by model  $i$ .  $N'_i(\hat{k}) = \lceil \frac{\hat{k}}{b_{i, \max}} \rceil$  is the total number of batches required to serve  $\hat{k}$  users with model  $i$ . Furthermore, the second term  $V \left( \sum_{l=l_i+1}^{L_i} S'_{i,l} \right) = V_1 \left( \sum_{l=l_i+1}^{L_i} S'_{i,l} \right) + V_2 \left( \sum_{l=l_i+1}^{L_i} S'_{i,l} \right)$  is the total model loading time for the top  $L_i - l_i$  layers of model  $i$ , which are not shared with model  $\hat{i}$ . It includes loading time from the disk to system memory and from system memory to GPU memory. Here,  $S'_{i,l}$  is the size of the  $l$ -th layer of

model  $i$  and  $l_0 = 0$ . Moreover, the last term  $\sum_{n=1}^{N'_i(\hat{k})} e(n, i, \hat{k})$  is the total inference computing time for  $\hat{k}$  users served in consecutive batches with model  $i$ . Here, based on Theorem 2,  $e(n, i, \hat{k})$  is given as

$$e(n, i, \hat{k}) = \mu_i \min \left\{ b_{i, \max}, \hat{k} - (n-1)b_{i, \max} \right\} + \beta_i, \quad (17)$$

where  $\min \left\{ b_{i, \max}, \hat{k} - (n-1)b_{i, \max} \right\}$  is the batch size of batch  $n$ , determined by  $b_{i, \max}$  and the remaining number of users after scheduling the first  $n-1$  batches.

2) *Number of served users in a model cluster:* We use  $g_m(i, \tau_2)$  to represent the maximum number of users that can be served with the first  $i$  models in  $\mathcal{I}_m$  within the first  $\tau_2$  time slots of a time period consisting of  $\tilde{\tau}$  time slots in total. Based on Theorem 2 and Corollary 1,  $g_m(i, \tau_2)$  is expressed as

$$g_m(i, \tau_2) = \begin{cases} \Gamma_m(i, \tau_2), & \text{if } \tau_2 \in [1, \tilde{\tau}], i \in \mathcal{I}_m \\ 0, & \text{if } i = 0 \text{ or } \tau_2 = 0, \end{cases} \quad (18)$$

where  $\Gamma_m(i, \tau_2)$  is provided in (19) at the bottom of next page.

In  $\Gamma_m(i, \tau_2)$ , three terms correspond to three cases considered for model  $i$  in cluster  $m$ . For the first term  $\gamma_m(i, \tau_2)$ , we consider that model  $\hat{i}$  is the preceding model of model  $i$ , and model  $i$  is allocated to  $\hat{\tau}$  time slots to serve  $q_m(\hat{i}, i, \hat{\tau})$  users, where  $g_m(\hat{i}, \tau_2 - \hat{\tau})$  is the number of users served by the first  $\hat{i}$  models in cluster  $m$  within  $\tau_2 - \hat{\tau}$  time slots. For the second term  $q_m(\hat{i} = 0, i, \hat{\tau} = \tau_2)$ , we consider that model  $i$  does not have a preceding model (i.e.,  $\hat{i} = 0$ ), and all  $\tau_2$  time slots are allocated to model  $i$  (i.e.,  $\hat{\tau} = \tau_2$ ). For the third term  $g_m(\hat{i} = i-1, \tau_2)$ , we consider that there is no need to load model  $i$ , leading to  $g_m(i, \tau_2) = g_m(i-1, \tau_2)$ , as the number of served users remains the same.

3) *Number of served users across all model clusters:* We use  $f(m, \tau_1)$  to represent the maximum number of users that can be served with the first  $m$  model clusters within the first  $\tau_1$  time slots. Based on Theorem 3 and Corollary 1,  $f(m, \tau_1)$  can be given as

$$f(m, \tau_1) = \begin{cases} \max_{\tilde{\tau} \in [0, \tau_1]} \left\{ f(m-1, \tau_1 - \tilde{\tau}) + g_m(I_m, \tilde{\tau}) \right\}, & \text{if } \tau_1 \in [1, T], m \in \mathcal{M}, \\ 0, & \text{if } m = 0 \text{ or } \tau_1 = 0, \end{cases} \quad (20)$$

where  $g_m(I_m, \tilde{\tau})$ , as recalled, represents the maximum number of users that can be served by the first  $I_m$  models in cluster  $m$  in multiple consecutive batches within  $\tilde{\tau}$  time slots.

After traversing all clusters and time slots in  $f(m, \tau_1)$ , the maximum number of served users can be determined by

$$U(\tilde{\mathbf{X}}^*) = f(M, T), \quad (21)$$

where  $\tilde{\mathbf{X}}^* = \bigcup_{n=1}^N \tilde{\mathbf{X}}_n^*$  is the determined user scheduling in the BLS case, with  $\tilde{x}_{n,k}^* \in \tilde{\mathbf{X}}_n^*$ , which will be elaborated next.

4) *Determination of  $\tilde{\mathbf{X}}^*$* : The user scheduling decision corresponding to  $U(\tilde{\mathbf{X}}^*)$  can be obtained by Algorithm 1. In the outer loop (Line 2 to 28) over cluster  $m$ , we focus on identifying the maximum number of users served by models in cluster  $m$ , represented by  $g_m(I_m, \tilde{\tau}^*)$ . We determine the corresponding assigned time slots  $\tilde{\tau}^*$  for cluster  $m$  in Line 6 within the remaining  $\tau_1$  time slots by traversing the clusters in reverse order. In the inner loop (Line 8 to 26) over model  $i$  in cluster  $m$ , we first identify the optimal preceding model  $\hat{i}^*$  for model  $i$  in cluster  $m$  along with the corresponding assigned model slots  $\hat{\tau}^*$  for model  $i$  in Line 12 in the reverse order of models in  $\mathcal{I}_m$ . Then, we compute the maximum number of users,  $\hat{k}^*$ , that can be served using model  $i$  within  $\hat{\tau}^*$  time slots, assuming model  $i$  is loaded after  $\hat{i}^*$  in Line 13. Next, we calculate the number of batches for serving  $\hat{k}^*$  users with model  $i$  in Line 14. Finally, we schedule the first  $\hat{k}^*$  users in  $K_i$  across multiple batches and insert these batches at the start of  $\tilde{\mathbf{X}}_N^*$  from Line 15 to 24. Note that there may be more than one user scheduling decision that can achieve the maximum throughput, and Algorithm 1 produces one of such solutions by starting from the maximum task throughput and identifying the feasible maximum number of served users in each loop.

5) *Algorithm outline and analysis*: To solve  $\mathcal{P}1$  in the BLS case, we summarize the entire process in Algorithm 2, which calculates  $\tilde{\mathbf{X}}^*$  from Algorithm 1 and derives the optimal bandwidth allocation  $\tilde{y}_{n,k}^* \in \tilde{\mathbf{Y}}^*$  according to (12). Furthermore, we establish the following theorem for Algorithm 2.

**Theorem 4.** *The proposed Algorithm 2 achieves the optimal solution to  $\mathcal{P}1$  in the BLS case with a polynomial-time computational complexity  $O(MI^2K)$ .*

*Proof.* We begin by proving that Algorithm 2 achieves the optimal solution in the BLS case. First, based on Theorem 2, (15) computes the maximum number of served users requesting model  $i$  within the scheduled  $\hat{\tau}$  time slots, and (18) computes the maximum number of served users requesting models in  $\mathcal{I}_m$ . Second, by Corollary 1, (20) computes the maximum number of served users requesting models in  $\mathcal{I}$ . Third, the proposed algorithm adopts a DP scheme, which guarantees optimality by traversing all models within each model cluster and across all model clusters using (18) and (20), respectively. Therefore,  $\tilde{\mathbf{X}}^*$ , obtained in Line 24 of Algorithm 2, is the optimal user scheduling for  $\mathcal{P}2$ . Furthermore, based on Proposition 1 and Theorem 1,  $\tilde{\mathbf{X}}^*$  and  $\tilde{\mathbf{Y}}^*$  are the optimal user scheduling and spectrum bandwidth allocation for  $\mathcal{P}1$ , respectively.

Next, we show the time complexity of Algorithm 2 is  $O(MI^2K)$ . First, the time complexity from Line 2 to 22 in Algorithm 2 is  $O(TMI^2K + T^2MI^2 + T^2M) =$

---

**Algorithm 1:** Recursive Algorithm for Determining User Scheduling

---

**Input:**  $q_m(\hat{i}, i, \hat{\tau})$ ,  $g_m(i, \tau_2)$ , and  $f(m, \tau_1)$  calculated in Algorithm 2.  $\mathcal{I}_m$  and  $\mathcal{K}$ .

**Output:**  $\tilde{\mathbf{X}}^*$ .

```

1 Initialize:  $m = M$ ,  $\tau_1 = T$ ,  $N = 0$ ,  $\tilde{\mathbf{X}}_N^* = \emptyset$ .
2 while  $m > 0$  do
3   if  $\tau_1 \leq 0$  then
4     Break.
5   end
6    $\tilde{\tau}^* = \arg \max_{\tilde{\tau} \in [0, \tau_1]} \{f(m-1, \tau_1 - \tilde{\tau}) + g_m(I_m, \tilde{\tau})\}$ .
7    $i = I_m$  and  $\tau_2 = \tilde{\tau}^*$ .
8   while  $i > 0$  do
9     if  $\tau_2 \leq 0$  then
10      Break.
11    end
12     $\{\hat{i}^*, \hat{\tau}^*\} = \arg \max_{\hat{i}, \hat{\tau}} \Gamma_m(i, \tau_2)$ .
13     $\hat{k}^* = q_m(\hat{i}^*, i, \hat{\tau}^*)$ .
14     $N'_i(\hat{k}^*) = \lceil \frac{\hat{k}^*}{b_{i, \max}} \rceil$ .
15    if  $N \neq 0$  then
16       $n = N$ .
17      for  $1 \leq n \leq N$  do
18         $\tilde{\mathbf{X}}_{n+N'_i(\hat{k}^*)}^* = \tilde{\mathbf{X}}_n^*$ .  $n = n + 1$ .
19      end
20    end
21    for  $1 \leq n \leq N'_i(\hat{k}^*)$  do
22       $\tilde{\mathbf{X}}_n^* = \{\tilde{x}_{n,k}^*\}$ , where  $k \in \mathcal{K}_i$  and  $k \in$ 
23         $\left\{ (n-1)b_{i, \max} + 1, \dots, (n-1)b_{i, \max} \right\}$ 
24         $\left\{ + \min \{b_{i, \max}, \hat{k}^* - (n-1)b_{i, \max} \} \right\}$ 
25         $n = n + 1$ .
26    end
27     $N = N + N'_i(\hat{k}^*)$ ,  $i = \hat{i}^*$ ,  $\tau_2 = \tau_2 - \hat{\tau}^*$ .
28  end
29  $\tilde{\mathbf{X}}^* = \bigcup_{n=1}^N \tilde{\mathbf{X}}_n^*$ .
```

---

$O(MI^2K)$ , where  $O(TMI^2K)$  is the time complexity of Line 2 to 10,  $O(T^2MI^2)$  accounts for Line 11 to 17, and  $O(T^2M)$  corresponds to Line 18 to 22 in Algorithm 2. Second, the time complexity of Algorithm 1

$$\Gamma_m(i, \tau_2) = \max \left\{ \begin{aligned} & \gamma_m(i, \tau_2) = \max_{\hat{i} \in (0, i-1], \hat{i} \in \mathcal{I}_m, \hat{\tau} \in [1, \tau_2]} \left\{ \begin{aligned} & g_m(\hat{i}, \tau_2 - \hat{\tau}) \mid g_m(\hat{i}, \tau_2 - \hat{\tau}) > 0, \\ & + q_m(\hat{i}, i, \hat{\tau}) \mid g_m(\hat{i}, \tau_2 - \hat{\tau}) \neq g_m(\hat{i}-1, \tau_2 - \hat{\tau}) \end{aligned} \right\}, \\ & q_m(\hat{i} = 0, i, \hat{\tau} = \tau_2), \quad g_m(i-1, \tau_2) \end{aligned} \right\} \quad (19)$$



---

**Algorithm 2:** DP-based Algorithm
 

---

**Input:**  $\mathcal{K}, \mathcal{I}, \mathcal{M}, T$ .

**Output:**  $\tilde{\mathbf{X}}^*, \tilde{\mathbf{Y}}^*, U(\tilde{\mathbf{X}}^*)$ .

```

1 Initialize: Set  $q_m(\hat{i}, i, \hat{\tau})$ ,  $g_m(i, \tau_2)$ ,  $f(m, \tau_1)$  to 0.
2 for  $1 \leq m \leq M$  do
3   for  $0 \leq i \leq I_m$  do
4     for  $0 \leq \hat{\tau} \leq T$  do
5       for  $0 \leq \hat{i} \leq i - 1$  do
6         Calculate  $q_m(\hat{i}, i, \hat{\tau})$  from (15).
7       end
8     end
9   end
10 end
11 for  $1 \leq m \leq M$  do
12   for  $0 \leq i \leq I_m$  do
13     for  $0 \leq \tau_2 \leq T$  do
14       Calculate  $g_m(i, \tau_2)$  from (18).
15     end
16   end
17 end
18 for  $0 \leq m \leq M$  do
19   for  $0 \leq \tau_1 \leq T$  do
20     Calculate  $f(m, \tau_1)$  from (20).
21   end
22 end
23  $U(\tilde{\mathbf{X}}^*) = f(M, T)$ .
24 With  $q_m(\hat{i}, i, \hat{\tau})$ ,  $g_m(i, \tau_2)$ , and  $f(m, \tau_1)$ , calculate
     $\tilde{\mathbf{X}}^*$  from Algorithm 1.
25 Calculate  $\tilde{\mathbf{Y}}^*$  using  $\tilde{y}_{n,k}^* = \frac{\tilde{x}_{n,k}^* D_k}{B \bar{R}_k \sum_{k' \in \mathcal{K}} \frac{\tilde{x}_{n,k'}^* D_{k'}}{B \bar{R}_{k'}}}$ .
  
```

---

is  $O(MT + MI(IT + 1 + K)) = O(TMI^2 + MIK)$ , where  $O(MT)$  corresponds to Line 6 in Algorithm 1,  $O(MI(IT + 1 + K))$  accounts for Line 12, Line 13, and Line 15 to 24 in Algorithm 1. Note that the time complexity of Line 13 in Algorithm 1 is  $O(1)$ , as the value of  $q_m(\hat{i}^*, i, \hat{\tau}^*)$  has been computed in Algorithm 2 before executing Algorithm 1. Third, the time complexity of computing  $\tilde{\mathbf{Y}}$  in Line 25 of Algorithm 2 is  $O(K)$ . Therefore, the total time complexity of Algorithm 2 is  $O(MI^2K)$ , which completes the proof.  $\square$

**Remark 4. Optimality in independent model loading:** Theorem 4 remains valid for multi-user edge inference systems when parameter sharing is not exploited. In such scenarios, all parameter blocks of an AI model are fully loaded during the model loading step of an inference batch, and Theorem 3 and Corollary 1 continue to ensure optimality. Therefore, while Algorithm 2 is designed for the BLS case, it also guarantees an optimal solution for independent model loading with the same computational time complexity.

## VI. GENERAL CASE

This section addresses the general case of  $\mathcal{P}1$ . In the general case, we do not impose any assumptions on the shared

parameter blocks across models; these blocks can appear at arbitrary positions within a model rather than forming consecutive bottom layers. Although  $\mathcal{P}1$  can still be decoupled into solving  $\mathcal{P}2$  followed by calculating the optimal spectrum bandwidth allocation for any given user scheduling solution using (12) in the general case, Theorem 3 and Corollary 1, which hold in the BLS case, do not apply to  $\mathcal{P}2$  in the general case. As  $\mathcal{P}2$  is NP-hard, finding the optimal model loading order across batches is computationally intractable in polynomial time. Consequently, finding the optimal user scheduling in the general case is not computationally feasible. Therefore, we will develop a heuristic algorithm to obtain a sub-optimal user scheduling solution to  $\mathcal{P}2$  in the general case and compute the corresponding optimal spectrum bandwidth allocation.

Our heuristic employs a greedy procedure. Specifically, we aim to select the next model with the maximum number of users served per unit time slot in each iteration. Given that model  $i'$  is loaded before model  $i$ , the number of users served by model  $i$  per unit time slot within  $\tau$  time slots is given by

$$v(i', i, \tau) = \begin{cases} \max \left\{ \frac{k}{\tau} \mid \phi(i', i, \tau) \leq \tau \Delta \tau \right\}, & \text{if } \tau \in [1, T], i \in \mathcal{I}, \\ 0, & \text{if } i = 0 \text{ or } \tau = 0. \end{cases} \quad (22)$$

Analogous to (16),  $\phi(i', i, \tau)$  is the total inference latency for  $k$  users with model  $i$  loaded after model  $i'$  in the general case, which is

$$\begin{aligned} \phi(i', i, \tau) &= \sum_{n=1}^{N'_i(k)} p_{z(n, i, k)} + V \left( \sum_{j \in \mathcal{J}_i \setminus \mathcal{J}_{i'}} S_j \right) \\ &+ \sum_{n=1}^{N'_i(k)} e(n, i, k), k \in [0, |\mathcal{K}_i|], i' \in \mathcal{I} \setminus \{i\}, \end{aligned} \quad (23)$$

where  $\sum_{j \in \mathcal{J}_i \setminus \mathcal{J}_{i'}} S_j$  is the data size of parameter blocks of model  $i$  but not of model  $i'$ , which need to be loaded from the disk to system memory and then to GPU memory in the general case.  $\mathcal{J}_i$  is the set of parameter blocks of model  $i$ , with  $\mathcal{J}_0 = \emptyset$ .

The proposed algorithm is outlined in Algorithm 3. In the first loop over  $\tau'$ , we find the next model  $i^*$  and the corresponding assigned time slots  $\tau^*$  by identifying the maximum number of users served per unit time slot in Line 6. Next, we calculate the number of users served by model  $i^*$  within  $\tau^*$  time slots, denoted by  $k^*$ , in Line 10. Then, in Line 11, we calculate the number of batches required to serve  $k^*$  users with model  $i^*$ . Finally, from Line 12 to Line 15, we schedule the first  $k^*$  users in  $\mathcal{K}_i$  into multiple batches according to Theorem 2.1, appending these batches to the end of  $\mathbf{X}_N$ . Algorithm 3 exhibits a polynomial-time complexity as follows.

**Theorem 5.** The proposed Algorithm 3 solves  $\mathcal{P}1$  with a time complexity of  $O(K)$ .

*Proof.* The time complexity of Line 6 in Algorithm 3 is  $O(I)$ , and that of Line 12 to 15 is  $O(K)$ . Additionally, the time complexity of Line 19 in Algorithm 3 is  $O(K)$ . Therefore, the total time complexity of Algorithm 3 is  $O(\max\{I, K\}) = O(K)$ .

**Algorithm 3: Greedy Algorithm**


---

**Input:**  $\mathcal{I}, \mathcal{K}, T$ .  
**Output:**  $\mathbf{X}, \mathbf{Y}, U(\mathbf{X})$ .

- 1 **Initialize:**  $\mathcal{I}' = \mathcal{I}, \tau' = 0, N = 0, \tilde{\mathbf{X}}_N^* = \emptyset, i' = 0$ .
- 2 **while**  $\tau' < T$  **do**
- 3   **if**  $\mathcal{I}' = \emptyset$  **then**
- 4     **Break.**
- 5   **end**
- 6    $\{i^*, \tau^*\} = \arg \max_{i \in \mathcal{I}', \tau \in (0, T - \tau']}$   $\{v(i', i, \tau)\}$
- 7   **if**  $i^*$  *does not exist* **then**
- 8     **Break.**
- 9   **end**
- 10    $k^* = v(i', i^*, \tau^*) \tau^*$ .
- 11    $N'_i(k^*) = \lceil \frac{k^*}{b_{i^*, \max}} \rceil$ .
- 12   **for**  $1 \leq n \leq N'_i(k^*)$  **do**
- 13      $\mathbf{X}_{n+N} = \{x_{n,k}\}$ , where  $k \in \mathcal{K}_i$  and  $k \in$   
 $\left\{ (n-1)b_{i^*, \max} + 1, \dots, (n-1)b_{i^*, \max} \right\}$   
 $\left\{ + \min \{b_{i^*, \max}, k^* - (n-1)b_{i^*, \max} \} \right\}$ .
- 14      $n = n + 1$ .
- 15   **end**
- 16    $i' = i^*, \mathcal{I}' = \mathcal{I}' \setminus \{i^*\}, \tau' = \tau' + \tau^*,$   
 $N = N + N'_i(k^*)$ .
- 17 **end**
- 18  $\mathbf{X} = \bigcup_{n=1}^N \mathbf{X}_n$  and calculate  $U(\mathbf{X})$ .
- 19 Calculate  $\mathbf{Y}$  using (12).

---

Here, the equality holds since one AI model in the model library is requested by at least one user, which completes the proof.  $\square$

## VII. NUMERICAL RESULTS

### A. Simulation Setup

In our simulation settings, the coverage radius of the edge server is 250 m, with  $K$  users evenly distributed in the coverage area of the edge server, where  $K$  is chosen from  $\{60, 70, 80, 90, 100\}$ .  $P_k$ ,  $N_0$ , and  $\alpha$  of  $R_k$  in (6) are set to  $5 \times 10^{-9}$  Watt/Hz, -174 dBm/Hz, and 4, respectively. The total bandwidth  $B$  ranges from  $\{10, 50, 100, 200, 300, 400\}$  MHz. The latency requirement  $\bar{T}$  varies between 500 and 900 ms, with each time slot lasting 10 ms.

An AI model library is constructed using three model structures: ResNet-18, ResNet-34, and ResNet-50. We first pre-train the three models on CIFAR100 and then fine-tune the pre-trained models on CIFAR10. In the BLS case, the bottom layer freezing technique is applied to fine-tune 50 AI models. The fine-tuned models share consecutive bottom layers with the corresponding pre-trained model of the same model structure. As a result, models fine-tuned from the same pre-trained model are grouped into a single model cluster and share bottom layers with the other models within the same cluster. In the general case, 25 AI models fine-tuned from the pre-trained model share parameter blocks at arbitrary positions. Moreover, in both the BLS and general cases, models share an average

of  $\theta = \{75\%, 80\%, 85\%, 90\%, 95\%\}$  of layers with the corresponding pre-trained models. For different values of  $\theta$ , compared with full-parameter fine-tuning, the inference accuracy changes, on average, by  $\{0.03\%, -0.03\%, 0.09\%, 0.19\%, -0.63\%\}$  in the BLS case and by  $\{0.02\%, -0.42\%, -0.49\%, -0.09\%, -1.15\%\}$  in the general case, with positive values indicating accuracy enhancement by overcoming overfitting. The dataset for inference computing is CIFAR10, where the original data size is resized from  $32 \times 32 \times 3$  to  $128 \times 128 \times 3$  to simulate higher-resolution data uploading.

Three algorithms are compared in the simulation.

- **PartialLoading Spec.** Algorithm 2, designed to obtain the optimal solution in the BLS case.
- **PartialLoading Gen.** Algorithm 3, developed for the general case. It is also applicable to the BLS case.
- **Independent Loading.** The *optimal* user scheduling and spectrum bandwidth allocation policy for  $\mathcal{P}1$  without considering parameter sharing. As noted in Remark 4, Algorithm 2 also produces the optimal solution when parameter sharing is not considered. This baseline serves as the performance upper bound for traditional schemes without exploiting parameter sharing.

We consider Rayleigh fading channels in the simulations. The results are averaged from  $10^3$  channel realizations. To evaluate the performance of the proposed approaches, we use the served user ratio as the metric, which is defined as the ratio of the number of users successfully served to the total number of users subject to the latency constraints. This metric is normalized to  $[0, 1]$ . In addition, we use a Linux edge server with configurations consistent with Fig. 1.

### B. Performance in the BLS Case

We first evaluate the performance of the proposed approaches in the BLS case. Although PartialLoading Gen is designed for the general case, it can also be applied to the BLS case and is therefore included as a benchmark.

In Fig. 6, we compare the performance by varying the total bandwidth  $B$ , number of users  $K$ , latency constraint  $\bar{T}$ , and layer sharing ratio  $\theta$ . Fig. 6(a) demonstrates that increasing the value of  $B$  can increase the served user ratio. This is because the uploading time in (6) decreases as more communication resources are allocated to users. Notably, when  $B$  ranges from 10 to 100 MHz, the served user ratio increases significantly with growing bandwidth. However, the increasing trend slows when  $B$  is larger than 100 MHz. It illustrates that the performance bottleneck of the proposed framework lies in the data uploading when communication resources are limited. In contrast, the bottleneck shifts to model loading and inference computing when communication resources are abundant. This insight can guide network operators in allocating communication and computing resources effectively to balance performance across different stages of the framework. Moreover, both PartialLoading Spec and PartialLoading Gen outperform Independent Loading. Specifically, PartialLoading Spec and PartialLoading Gen improve the served user ratio by about 24% and 16%, respectively, on average. Note that PartialLoading Spec obtains the optimal solution in the BLS case.

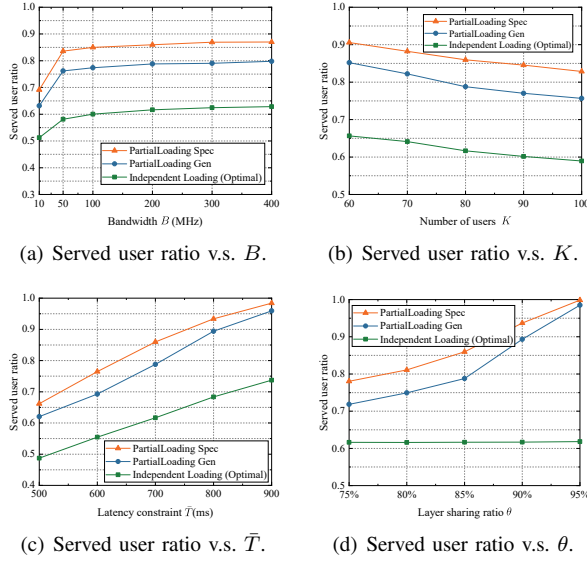


Fig. 6. Served user ratio in the BLS case, where  $B$ ,  $K$ ,  $\bar{T}$ , and  $\theta$  are respectively set to 200 MHz, 80, 700 ms, and 85% by default.

As a result, the small performance gap between PartialLoading Spec and PartialLoading Gen indicates that PartialLoading Gen also performs well.

Fig. 6(b) illustrates the served user ratio for varying  $K$  in the edge network. As  $K$  increases, the served user ratio decreases due to the communication and computing resource limitations of the edge server, indicating that the system's capacity has been reached. Nevertheless, the edge server can still serve nearly 83% and 76% of users with PartialLoading Spec and PartialLoading Gen, respectively. This demonstrates that the proposed approaches still perform well even with a high density of users. Moreover, PartialLoading Spec and PartialLoading Gen outperform Independent Loading, improving the served user ratio by about 24% and 18%, respectively.

In Fig. 6(c), we study the impact of  $\bar{T}$  on the performance. As expected, the served user ratio increases proportionally with  $\bar{T}$ . This figure also shows a nearly linear relationship between the served user ratio and  $\bar{T}$ . Furthermore, PartialLoading Spec and PartialLoading Gen improve the served user ratio by about 22% and 18%, respectively, over Independent Loading.

We finally analyze the effect of  $\theta$  in Fig. 6(d). PartialLoading Spec and PartialLoading Gen enhance the served user ratio by about 26% and 21% than Independent Loading. Besides, the served user ratio increases at a faster rate as  $\theta$  grows. The reason is that the size of the top layers in the utilized models is generally larger than that of the bottom layers. In the BLS case, a higher  $\theta$  implies that more top layers are shared and do not need to be loaded. Therefore, the increasing trend of the served user ratio becomes more pronounced.

### C. Performance in the General Case

In this subsection, we evaluate the performance of PartialLoading Gen in the general case, where the shared parameter blocks can appear at arbitrary positions within AI models. Since PartialLoading Spec is specifically designed for the BLS

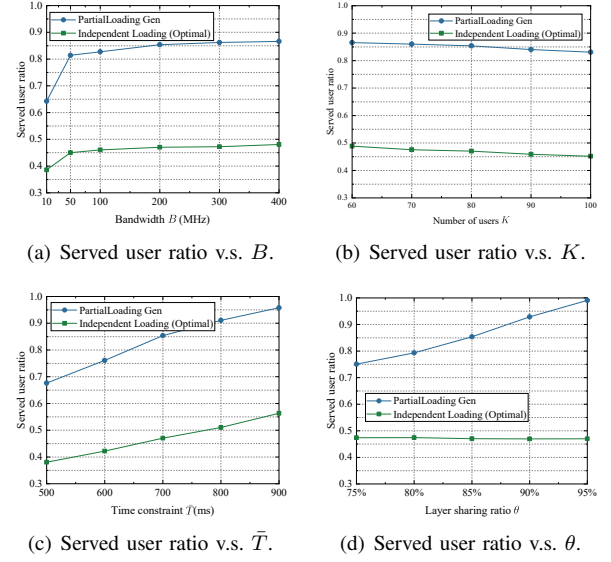


Fig. 7. Served user ratio in the general case, where the default values of  $B$ ,  $K$ ,  $\bar{T}$ , and  $\theta$  are the same as those in Fig. 6.

case, in this subsection, we only compare PartialLoading Gen with Independent Loading.

Similar to the above subsection, we analyze the served user ratio of PartialLoading Gen under varying values of  $B$ ,  $K$ ,  $\bar{T}$ , and  $\theta$  in Fig. 7. In Figs. 7(a), 7(b), 7(c), and 7(d), trends similar to those in Fig. 6 can be observed. In these figures, PartialLoading Gen improves the served user ratio by about 36%, 38%, 36%, and 39%, respectively, compared with Independent Loading. Moreover, in Fig. 7(d), due to the arbitrary position of shared parameter blocks, the increasing trend of the served user ratio remains stable as  $\theta$  grows.

### D. Ablation Study

This subsection evaluates the impact of spectrum bandwidth allocation on the served user ratio in the proposed framework through ablation experiments. Fig. 8 compares the performance of the proposed PartialLoading Spec and PartialLoading Gen algorithms with their counterparts under equal bandwidth allocation in both the BLS and general cases. In the setup of algorithms with equal bandwidth allocation, the total bandwidth  $B$  is evenly divided into  $r \in \{5, 10, 20\}$  sub-channels, with each sub-channel assigned to a single user. The bandwidth allocated to user  $k$  in batch  $n$  in (4) is updated to  $B'_{n,k} = x_{n,k}B_k$ , where  $B_k = \frac{B}{r}$ , satisfying  $\sum_{k \in K} x_{n,k}B_k \leq B$ ,  $\forall n \in \mathcal{N}$ . Then, the corresponding user scheduling is determined using  $B'_{n,k}$  for the BLS case and general case by PartialLoading Spec and PartialLoading Gen, respectively.

Figs. 8(a) and 8(b) demonstrate that the proposed PartialLoading Spec consistently outperforms the algorithms with equal bandwidth allocation across varying  $B$  and  $K$  in the BLS case. Specifically, in Fig. 8(a), PartialLoading Spec achieves average served user ratio improvements of approximately 9.9%, 16.9%, and 26.2% over the equal-bandwidth-allocation algorithms with  $r = 5$ ,  $r = 10$ , and  $r = 20$ , respectively.

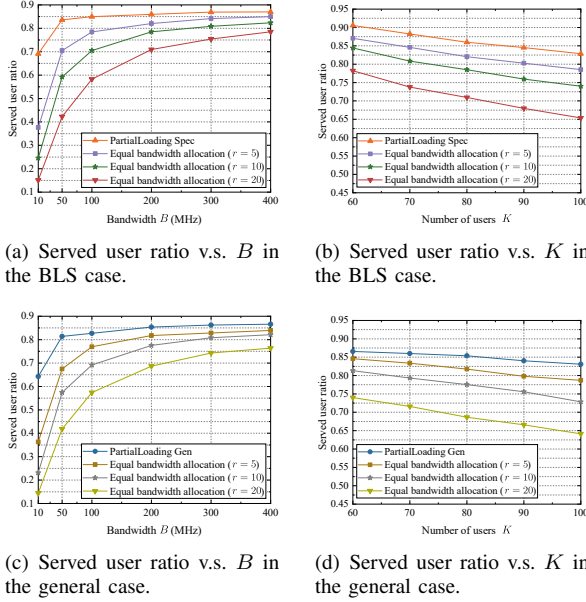


Fig. 8. Ablation study for bandwidth allocation, where the default values of  $B$ ,  $K$ ,  $\bar{T}$ , and  $\theta$  are the same as those in Fig. 6. In the equal-bandwidth-allocation algorithms,  $r$  represents the number of sub-channels into which the total bandwidth  $B$  is evenly divided, with each sub-channel assigned to a single user.

Similarly, in Fig. 8(b), the respective improvements are approximately 3.9%, 7.7%, and 15.2%.

Figs. 8(c) and 8(d) exhibit trends similar to those in Figs. 8(a) and 8(b), demonstrating PartialLoading Gen consistently outperforms the equal-bandwidth-allocation algorithms across varying  $B$  and  $K$  in the general case. In Fig. 8(c), the respective improvements are about 9.6%, 16.1%, and 25.6%, while in Fig. 8(d), they are about 3.4%, 7.7%, and 16.1%.

### E. Running Time Comparisons

In this subsection, we evaluate the efficiency of the proposed algorithms by comparing the served user ratio and average running time of the proposed algorithms against the exhaustive search. To ensure the feasible execution of the exhaustive search, we consider a small problem scale with  $K$ ,  $I$ , and  $\bar{T}$  set to 20, 5, and 200 ms, respectively.

Fig. 9(a) compares the performance among PartialLoading Gen, PartialLoading Spec, and the exhaustive search in the BLS case. It illustrates that the served user ratio of PartialLoading Spec matches that of the exhaustive search, validating the optimality of the PartialLoading Spec. However, the running time of PartialLoading Spec is only about 5.6 ms, making it nearly 3,681 times faster than the exhaustive search, where the time complexity of the exhaustive search grows exponentially with  $I$  and  $\bar{T}$ . Additionally, the served user ratio of PartialLoading Gen is only about 3.7% lower than the exhaustive search, yet it achieves approximately 3,749 times faster execution.

Similarly, in Fig. 9(b), we compare the performance between PartialLoading Gen and the exhaustive search in the general case. The served user ratio of PartialLoading Gen only

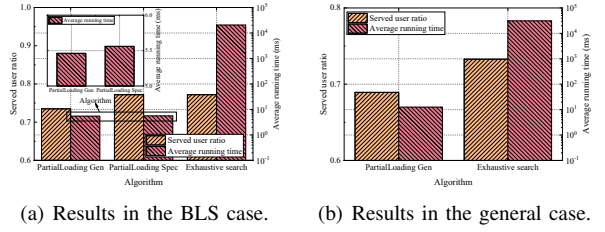


Fig. 9. Comparisons of served user ratio and average running time for different algorithms, with default values of  $B$ , and  $\theta$  as in Fig. 6.

degrades by about 4.3%. In contrast, it runs in only about 12.5 ms and accelerates the decision process by about 2,478 times.

## VIII. CONCLUSIONS

In this paper, we investigated the multi-user scheduling problem in parameter-sharing edge inference and proposed a PartialLoading framework to maximize inference task throughput. By leveraging shared parameter blocks to avoid redundant model loading across consecutive inference batches, the proposed framework optimizes task throughput by scheduling users and allocating spectrum bandwidth under latency and resource constraints. To simplify the solution process, we decomposed the original problem into two sub-problems, i.e., user scheduling and bandwidth allocation sub-problems, without compromising the optimality of the original problem. For the bandwidth allocation sub-problem, we derived the optimal closed-form expression for spectrum bandwidth allocation given user scheduling decisions. For the user-scheduling sub-problem, we first investigated the bottom-layer-sharing case and designed a dynamic programming-based algorithm to find the optimal solution in polynomial time. Then, we addressed the general case by developing a greedy heuristic procedure. Both the proposed algorithms demonstrated significant task throughput improvements compared with traditional frameworks without exploiting the parameter-sharing property. By pioneering user scheduling for parameter-sharing edge inference, we hope this work offers insights into further improving the efficiency of edge inference systems.

## APPENDIX

Before presenting the detailed proof, we introduce the following statements, which apply to Appendices B through E. A user scheduling sequence  $\mathbf{X} = \{\mathbf{X}_1, \dots, \mathbf{X}_N\}$  can be partitioned into multiple disjoint sub-sequences. Denote the  $a$ -th sub-sequence as  $\Pi_a = \{\mathbf{X}_n, \mathbf{X}_{n+1}, \dots, \mathbf{X}_{n'}\}$ , where  $1 \leq n \leq n' \leq N$ . Moreover, let  $c(\Pi_a)$  be the completion time of  $\Pi_a = \{\dots, \mathbf{X}_n\}$ , where  $c(\Pi_a) = \sum_{n'=1}^n t_{n'}$ .

### APPENDIX A PROOF OF PROPOSITION 1

We first prove the first statement, which is a necessary and sufficient condition. The proof of necessity is as follows. Given a user scheduling  $\mathbf{X}$ , suppose in the optimal bandwidth allocation  $\mathbf{Y}$  corresponding to  $\mathbf{X}$ , there exists a batch  $n$  such that  $t_n^{\text{dp}}$  is larger than its minimum possible value. By

adjusting  $y_{n,k}$  to minimize  $t_n^{\text{up}}$ , the uploading time could be reduced, allowing more users to be served. This adjustment contradicts the optimality of the assumption. Therefore, the optimal  $y_{n,k}$  corresponding to any  $x_{n,k}$  in batch  $n$  must minimize  $t_n^{\text{up}}$ . The proof of sufficiency is as follows. Given a user scheduling  $\mathbf{X}$ , suppose there exists a spectrum bandwidth allocation policy  $\mathbf{Y}$  that minimizes  $t_n^{\text{up}}$  under  $\mathbf{X}$ , while  $\mathbf{Y}$  is not the optimal policy under  $\mathbf{X}$ . This implies that the total uploading time of the users in  $\mathbf{X}$  with the optimal bandwidth allocation is larger than that with  $\mathbf{Y}$ . However, this contradicts the definition of the optimal spectrum bandwidth allocation, as  $\mathbf{Y}$  yields a shorter total uploading time for the users in  $\mathbf{X}$  while maintaining the total model loading and inference computing time unchanged. Thus, more users could be served if bandwidth were allocated according to  $\mathbf{Y}$ . Consequently, the bandwidth allocation policy that minimizes  $t_n^{\text{up}}$  is optimal for the users in  $\mathbf{X}$ . This completes the proof of the first statement.

Now we prove the second statement and derive (11) and (12). Let  $\dot{\mathcal{K}}_n = \{k \mid x_{n,k} = 1\}$  be the set of scheduled users in batch  $n$ .

If  $|\dot{\mathcal{K}}_n| \geq 1$ , then  $t_n^{\text{up}} = \max_{k \in \dot{\mathcal{K}}} \{t_{n,k}^{\text{up}}\} = \max_{k \in \dot{\mathcal{K}}_n} \{t_{n,k}^{\text{up}}\} \geq \frac{\sum_{k \in \dot{\mathcal{K}}_n} t_{n,k}^{\text{up}}}{|\dot{\mathcal{K}}_n|} \geq |\dot{\mathcal{K}}_n| \sqrt{\prod_{k \in \dot{\mathcal{K}}_n} t_{n,k}^{\text{up}}}$ . The second inequality holds with

equality if and only if  $t_{n,k}^{\text{up}} = t_{n,k'}^{\text{up}}$  for all  $k, k' \in \dot{\mathcal{K}}_n$ , i.e., all users in  $\dot{\mathcal{K}}_n$  have the same uploading time. On the one hand, for all  $k \in \dot{\mathcal{K}}_n$ , since  $t_{n,k}^{\text{up}} = \frac{D_k}{y_{n,k} B \bar{R}_k}$ , the equality  $\frac{\sum_{k \in \dot{\mathcal{K}}_n} t_{n,k}^{\text{up}}}{|\dot{\mathcal{K}}_n|} = |\dot{\mathcal{K}}_n| \sqrt{\prod_{k \in \dot{\mathcal{K}}_n} t_{n,k}^{\text{up}}}$  holds if and only if

$y_{n,k} = \frac{D_k}{B \bar{R}_k \sum_{k' \in \dot{\mathcal{K}}_n} \frac{D_{k'}}{B \bar{R}_{k'}}} = \frac{D_k}{B \bar{R}_k \sum_{k' \in \dot{\mathcal{K}}} \frac{x_{n,k'} D_{k'}}{B \bar{R}_{k'}}}$ . Substituting this into  $t_{n,k}^{\text{up}}$ , the minimum uploading time is given by  $t_{n,\min}^{\text{up}} = |\dot{\mathcal{K}}_n| \sqrt{\prod_{k \in \dot{\mathcal{K}}_n} t_{n,k}^{\text{up}}} = t_{n,k}^{\text{up}} = \frac{D_k}{B \bar{R}_k \sum_{k' \in \dot{\mathcal{K}}} \frac{x_{n,k'} D_{k'}}{B \bar{R}_{k'}}} =$

$\sum_{k' \in \dot{\mathcal{K}}} \frac{x_{n,k'} D_{k'}}{B \bar{R}_{k'}}$ . On the other hand, for all  $k \in \mathcal{K} \setminus \dot{\mathcal{K}}_n$ , since  $x_{n,k} = 0$ , we have  $y_{n,k} = 0$ , which is constrained by (10e).

If  $|\dot{\mathcal{K}}_n| = 0$ , it indicates that no users are scheduled in batch  $n$ , i.e.,  $x_{n,k} = 0, \forall k \in \mathcal{K}$ . Then,  $t_n^{\text{up}} = 0$  and  $y_{n,k} = 0$ , which are consistent with (11) and (12), respectively. This completes the proof of the second statement and concludes the proof.

## APPENDIX B

### PROOF OF THEOREM 2.1

The proof of the first statement proceeds as follows. Suppose the optimal user scheduling is  $\{\dots, \Pi_a, \Pi_{a+1}, \Pi_{a+2}, \Pi_{a+3}, \dots\}$ , where users in  $\Pi_a$  and  $\Pi_{a+2}$  request the same AI model  $i$ , users in  $\Pi_{a+3}$  request model  $i_3$ , and users in  $\Pi_{a+1} = \{\mathbf{X}_{n_1}, \dots, \mathbf{X}_{n_2}\}$  request at least one AI model other than model  $i$ . Moreover, let models  $i_1$  and  $i_2$  denote the models requested in batches  $n_1$  and  $n_2$ , respectively, where  $i_3 \neq i, i_1 \neq i$ , and  $i_2 \neq i$ . Now consider exchanging  $\Pi_{a+1}$  and  $\Pi_{a+2}$  in the original user scheduling. Denote the completion time of  $\Pi_{a+3}$  in the new user scheduling as  $c'(\Pi_{a+3})$ . We claim that

$c'(\Pi_{a+3}) \leq c(\Pi_{a+3})$ . The reasoning is as follows. In the original user scheduling  $\{\dots, \Pi_a, \Pi_{a+1}, \Pi_{a+2}, \Pi_{a+3}, \dots\}$ , the data size of parameter blocks that need to be loaded in the first batch of  $\Pi_{a+1}$ ,  $\Pi_{a+2}$ , and  $\Pi_{a+3}$  are, respectively:  $\sum_{j \in \mathcal{J}_{i_1} \setminus \mathcal{J}_i} S_j$ ,  $\sum_{j \in \mathcal{J}_i \setminus \mathcal{J}_{i_2}} S_j$ , and  $\sum_{j \in \mathcal{J}_{i_3} \setminus \mathcal{J}_i} S_j$ . In the new user scheduling  $\{\dots, \Pi_a, \Pi_{a+2}, \Pi_{a+1}, \Pi_{a+3}, \dots\}$ , the data size of parameter blocks that need to be loaded in the first batch of  $\Pi_{a+2}$ ,  $\Pi_{a+1}$ , and  $\Pi_{a+3}$  are, respectively: 0,  $\sum_{j \in \mathcal{J}_{i_1} \setminus \mathcal{J}_i} S_j$ , and  $\sum_{j \in \mathcal{J}_{i_3} \setminus \mathcal{J}_{i_2}} S_j$ . For any parameter block  $j \in \mathcal{J}_{i_3} \setminus \mathcal{J}_{i_2}$ , if  $j \in \mathcal{J}_i$ , then  $j \in \mathcal{J}_i \setminus \mathcal{J}_{i_2}$ , as  $j \notin \mathcal{J}_{i_2}$ . Conversely, if  $j \notin \mathcal{J}_i$ , then  $j \in \mathcal{J}_{i_3} \setminus \mathcal{J}_i$ , as  $j \in \mathcal{J}_{i_3}$ . Thus, for all  $j \in \mathcal{J}_{i_3} \setminus \mathcal{J}_{i_2}$ , we have  $j \in (\mathcal{J}_i \setminus \mathcal{J}_{i_2}) \cup (\mathcal{J}_{i_3} \setminus \mathcal{J}_i)$ . This implies that  $\mathcal{J}_{i_3} \setminus \mathcal{J}_{i_2} \subseteq (\mathcal{J}_i \setminus \mathcal{J}_{i_2}) \cup (\mathcal{J}_{i_3} \setminus \mathcal{J}_i)$ . Consequently, we have:  $\sum_{j \in \mathcal{J}_{i_1} \setminus \mathcal{J}_i} S_j + \sum_{j \in \mathcal{J}_i \setminus \mathcal{J}_{i_2}} S_j + \sum_{j \in \mathcal{J}_{i_3} \setminus \mathcal{J}_i} S_j \geq \sum_{j \in \mathcal{J}_{i_1} \setminus \mathcal{J}_i} S_j + \sum_{j \in \mathcal{J}_{i_3} \setminus \mathcal{J}_{i_2}} S_j$ . This inequality represents that fewer parameter blocks need to be loaded in the new user scheduling after exchanging  $\Pi_{a+1}$  and  $\Pi_{a+2}$  in the original user scheduling. As a result, the completion time  $c'(\Pi_{a+3})$  of  $\Pi_{a+3}$  in the new scheduling is reduced, implying that more users could be served. This contradicts the optimality of the assumption, completing the proof of the first statement.

Now, we prove the second statement. Suppose in the optimal user scheduling,  $\Pi_a = \{\mathbf{X}_{n_1}, \dots, \mathbf{X}_n, \dots, \mathbf{X}_{n_2}\}$  consists exclusively of batches loading model  $i$ , and users in  $\mathbf{X}_n$  are not the  $|\mathbf{X}_n|$  users with the lowest  $p_k$  in  $\mathcal{K}_i \setminus \bigcup_{n'=n_1}^{n-1} \dot{\mathcal{K}}_{n'}$ , where  $\dot{\mathcal{K}}_{n'}$  is the set of users scheduled in batch  $n'$ . Then, we can adjust  $\mathbf{X}_n$  by selecting the  $|\mathbf{X}_n|$  users with the lowest  $p_k$  from  $\mathcal{K}_i \setminus \bigcup_{n'=n_1}^{n-1} \dot{\mathcal{K}}_{n'}$ . This adjustment maintains  $t_n^{\text{load}}$  and  $t_n^{\text{comp}}$  unchanged, as  $t_n^{\text{load}}$  is 0, and  $t_n^{\text{comp}}$  depends only on the batch size  $|\mathbf{X}_n|$ . However, this adjustment reduces  $t_n^{\text{up}}$ . Consequently, more users could be served in batch  $n$  in the new user scheduling, which contradicts the optimality of the assumption. This completes the proof of the second statement and concludes the proof.

## APPENDIX C

### PROOF OF THEOREM 2.2

Suppose in the optimal user scheduling,  $\mathbf{X}_n$  and  $\mathbf{X}_{n+1}$  are non-empty, and users in  $\mathbf{X}_n$  and  $\mathbf{X}_{n+1}$  request the same model  $i$ . Additionally, assume that  $A(|\mathbf{X}_n| + 1) \leq Q$ , which indicates that batch  $n$  can accommodate more than  $|\mathbf{X}_n|$  users without exceeding the GPU memory capacity. Based on these assumptions, we can move some users from batch  $n+1$  to batch  $n$  within the memory constraint, resulting in  $\mathbf{X}_n$  and  $\mathbf{X}_{n+1}$  being updated to  $\mathbf{X}'_n$  and  $\mathbf{X}'_{n+1}$ . Denote the data uploading time of the users in  $\mathbf{X}'_n$  and  $\mathbf{X}'_{n+1}$  by  $t_n^{\text{up}}$  and  $t_{n+1}^{\text{up}}$ , respectively. Similarly, denote the total latency of the users in  $\mathbf{X}'_n$  and  $\mathbf{X}'_{n+1}$  by  $t'_n$  and  $t'_{n+1}$ , respectively. Since  $\mathbf{X}'_n \cup \mathbf{X}'_{n+1}$  schedules the same users as  $\mathbf{X}_n \cup \mathbf{X}_{n+1}$ , we can derive that  $t_n^{\text{up}} + t_{n+1}^{\text{up}} = t_n^{\text{up}} + t_{n+1}^{\text{up}}$  and  $\mu_i \sum_{x_{n,k} \in \mathbf{X}_n \cup \mathbf{X}_{n+1}} x_{n,k} = \mu_i \sum_{x_{n,k} \in \mathbf{X}'_n \cup \mathbf{X}'_{n+1}} x_{n,k}$ . If  $\mathbf{X}'_{n+1}$



becomes empty after the adjustment, then batch  $n + 1$  is eliminated, and the reduction in total latency is:  $t_n + t_{n+1} - t'_n - t'_{n+1} = \beta_i$ . Otherwise, if  $\mathbf{X}'_{n+1}$  is not empty after the adjustment, then the total latency remains unchanged, satisfying:  $t_n + t_{n+1} - t'_n - t'_{n+1} = 0$ . Therefore, in either case, the total latency follows that  $t_{n+1} + t_n \geq t'_{n+1} + t'_n$ . The inequality indicates that more users could be served when some users in batch  $n + 1$  are moved to batch  $n$  in the original user scheduling under the given assumption, which contradicts the optimality of the assumption. This completes the proof.

#### APPENDIX D PROOF OF THEOREM 3

Suppose the optimal user scheduling is  $\{\dots, \Pi_a, \Pi_{a+1}, \Pi_{a+2}, \Pi_{a+3}, \dots\}$ , where all users in  $\Pi_a$  and  $\Pi_{a+2}$  request AI models from the same model cluster  $m$ , users in  $\Pi_{a+3}$  request models from model cluster  $m_3$ , and users in  $\Pi_{a+1} = \{\mathbf{X}_{n_1}, \dots, \mathbf{X}_{n_2}\}$  request models from at least one model cluster excluding cluster  $m$ . Additionally, the users in batch  $n_1$  and  $n_2$  request models from clusters  $m_1$  and  $m_2$ , respectively, where  $m_3 \neq m$ ,  $m_1 \neq m$ , and  $m_2 \neq m$ . Now, consider exchanging  $\Pi_{a+1}$  with  $\Pi_{a+2}$  in the original user scheduling. The completion time of  $\Pi_{a+3}$  in the new user scheduling is denoted by  $c'(\Pi_{a+3})$ .  $c'(\Pi_{a+3})$  follows that  $c'(\Pi_{a+3}) \leq c(\Pi_{a+3})$ . This is because the model loaded in the last batch of  $\Pi_a$  and the model loaded in the first batch of  $\Pi_{a+2}$  belong to the same model cluster, allowing bottom layer sharing across these two models. Such sharing could reduce the model loading time in the new user scheduling. Additionally, if  $m_2 = m_3$ ,  $c'(\Pi_{a+3})$  can be reduced further. Therefore, more users could be served, if  $\Pi_{a+1}$  is exchanged with  $\Pi_{a+2}$  in the original user scheduling  $\{\dots, \Pi_a, \Pi_{a+1}, \Pi_{a+2}, \Pi_{a+3}, \dots\}$ , which contradicts the optimality of the assumption. This completes the proof.

#### APPENDIX E PROOF OF COROLLARY 1

The first statement comes from the fact that the loading order of clusters does not impact loading time, as there is no layer sharing between models from different clusters.

The proof of the second statement is as follows. Consider  $\Pi = \{\dots, \Pi_a, \Pi_{a+1}, \Pi_{a+2}, \Pi_{a+3}, \dots\}$ , where users in  $\Pi$  request models from the same model cluster  $m$ , and let  $\mathcal{I}'_m$ , with  $I'_m$  models, be the set of all requested models in  $\Pi$ . Based on Theorem 2, users requesting the same model are batched in the same or consecutive batches. Thus, the total data size of layers to be loaded for  $\Pi$  is  $\sum_{i=1}^{I'_m} \sum_{l=1}^{L_i} S'_{i,l} - \dot{S}_m$ ,

where  $\sum_{i=1}^{I'_m} \sum_{l=1}^{L_i} S'_{i,l}$  is the total data size of all models in  $\mathcal{I}'_m$ , with  $S'_{i,l}$  being the data size of layer  $l$  of model  $i$ , and  $\dot{S}_m$  is the total data size of layers not requiring loading due to the bottom layer sharing. Since the first term is constant and independent of the model loading order, a model loading order for the users in  $\Pi$  that maximizes  $\dot{S}_m$  minimizes the total model loading time, thereby preserving the optimality of the user scheduling solution. Therefore, in the rest of the proof,

we aim to determine a model loading order for the users in  $\Pi$  that maximizes  $\dot{S}_m$ .

To analyze the impact of model loading order on  $\dot{S}_m$ , consider exchanging two neighboring sub-sequences, say  $\Pi_{a+1}$  and  $\Pi_{a+2}$ , in  $\Pi$ . This exchange affects the total data size of layers not requiring loading in  $\Pi_{a+1}$ ,  $\Pi_{a+2}$ , and  $\Pi_{a+3}$ . Suppose that users in  $\Pi_a$ ,  $\Pi_{a+1}$ ,  $\Pi_{a+2}$ , and  $\Pi_{a+3}$  request models  $i$ ,  $i_1$ ,  $i_2$ , and  $i_3$ , respectively, with  $l_{i_1} \leq l_{i_2}$  (The conclusion of the following derivation also holds when  $l_{i_1} \geq l_{i_2}$ , with the proof omitted.). The total data size of layers not requiring loading in  $\Pi_{a+1}$ ,  $\Pi_{a+2}$ , and  $\Pi_{a+3}$  before the exchange is

$$\dot{S}_{m,1} = \sum_{l=0}^{\min\{l_i, l_{i_1}\}} \tilde{S}_{m,l} + \sum_{l=0}^{l_{i_1}} \tilde{S}_{m,l} + \sum_{l=0}^{\min\{l_{i_2}, l_{i_3}\}} \tilde{S}_{m,l}, \text{ where}$$

$\tilde{S}_{m,l}$  is the data size of the  $l$ -th shared bottom layer of models in model cluster  $m$ , with  $l_0 = 0$  and  $\tilde{S}_{m,0} = 0$ . Here, the three terms in  $\dot{S}_{m,1}$  respectively correspond to the data size of layers not requiring loading in  $\Pi_{a+1}$ ,  $\Pi_{a+2}$ , and  $\Pi_{a+3}$ . After exchanging  $\Pi_{a+1}$  and  $\Pi_{a+2}$  in  $\Pi$ , the new user scheduling becomes  $\Pi' = \{\dots, \Pi_a, \Pi_{a+2}, \Pi_{a+1}, \Pi_{a+3}, \dots\}$ , and the total data size of layers not requiring loading in  $\Pi_{a+2}$ ,

$$\Pi_{a+1}, \text{ and } \Pi_{a+3} \text{ is } \dot{S}_{m,2} = \sum_{l=0}^{\min\{l_i, l_{i_2}\}} \tilde{S}_{m,l} + \sum_{l=0}^{l_{i_1}} \tilde{S}_{m,l} + \sum_{l=0}^{\min\{l_{i_1}, l_{i_3}\}} \tilde{S}_{m,l}. \text{ Then, the difference between } \dot{S}_{m,1} \text{ and } \dot{S}_{m,2}$$

$$\text{is given by } \dot{S}_{m,1} - \dot{S}_{m,2} = \sum_{l=0}^{\min\{l_i, l_{i_1}\}} \tilde{S}_{m,l} + \sum_{l=0}^{\min\{l_{i_2}, l_{i_3}\}} \tilde{S}_{m,l} - \sum_{l=0}^{\min\{l_{i_1}, l_{i_2}\}} \tilde{S}_{m,l} - \sum_{l=0}^{\min\{l_{i_1}, l_{i_3}\}} \tilde{S}_{m,l}.$$

Recalling that  $l_{i_1} \leq l_{i_2}$  in the assumption, we derive the following case analysis for this difference. If  $l_i \leq l_{i_3}$ , we analyze three possible sub-cases.

In the first sub-case, when  $l_i \leq l_{i_1}$ , we have  $\dot{S}_{m,1} - \dot{S}_{m,2} = \sum_{l=0}^{l_i} \tilde{S}_{m,l} + \sum_{l=0}^{\min\{l_{i_2}, l_{i_3}\}} \tilde{S}_{m,l} - \sum_{l=0}^{l_i} \tilde{S}_{m,l} - \sum_{l=0}^{\min\{l_{i_1}, l_{i_3}\}} \tilde{S}_{m,l} = \sum_{l=0}^{\min\{l_{i_2}, l_{i_3}\}} \tilde{S}_{m,l} - \sum_{l=0}^{\min\{l_{i_1}, l_{i_3}\}} \tilde{S}_{m,l}$ . Next, we can derive that

$$\dot{S}_{m,1} - \dot{S}_{m,2} = \begin{cases} \sum_{l=0}^{l_{i_3}} \tilde{S}_{m,l} - \sum_{l=0}^{l_{i_3}} \tilde{S}_{m,l} = 0, & \text{if } l_{i_3} \leq l_{i_1}, \\ \sum_{l=0}^{l_{i_3}} \tilde{S}_{m,l} - \sum_{l=0}^{l_{i_1}} \tilde{S}_{m,l} \geq 0, & \text{if } l_{i_1} \leq l_{i_3} \leq l_{i_2}, \\ \sum_{l=0}^{l_{i_2}} \tilde{S}_{m,l} - \sum_{l=0}^{l_{i_1}} \tilde{S}_{m,l} \geq 0, & \text{if } l_{i_2} \leq l_{i_3}. \end{cases} \quad (24)$$

Thus,  $\dot{S}_{m,1} - \dot{S}_{m,2} \geq 0$  consistently holds in this sub-case.

In the second sub-case, when  $l_{i_1} \leq l_i \leq l_{i_2}$ , we have

$$\dot{S}_{m,1} - \dot{S}_{m,2} = \sum_{l=0}^{l_{i_1}} \tilde{S}_{m,l} + \sum_{l=0}^{\min\{l_{i_2}, l_{i_3}\}} \tilde{S}_{m,l} - \sum_{l=0}^{l_i} \tilde{S}_{m,l} - \sum_{l=0}^{\min\{l_{i_1}, l_{i_3}\}} \tilde{S}_{m,l}. \text{ Since } l_{i_1} \leq l_i \leq l_{i_2} \text{ holds in this sub-case and } l_i \leq l_{i_3} \text{ holds in the case overall, we have } \sum_{l=0}^{\min\{l_{i_1}, l_{i_3}\}} \tilde{S}_{m,l} = \sum_{l=0}^{l_{i_1}} \tilde{S}_{m,l}. \text{ Furthermore, we can conclude}$$

that  $\dot{S}_{m,1} - \dot{S}_{m,2} \geq 0$  in the second sub-case. In the third sub-case, when  $l_{i_2} \leq l_i$ , we can derive that  $\dot{S}_{m,1} - \dot{S}_{m,2} = \sum_{l=0}^{l_{i_1}} \tilde{S}_{m,l} + \sum_{l=0}^{l_{i_2}} \tilde{S}_{m,l} - \sum_{l=0}^{l_{i_2}} \tilde{S}_{m,l} - \sum_{l=0}^{l_{i_1}} \tilde{S}_{m,l} = 0$  based on  $l_i \leq l_{i_3}$  for the overall case. Consequently,  $\dot{S}_{m,1} - \dot{S}_{m,2} \geq 0$  always holds when  $l_i \leq l_{i_3}$ . This indicates that scheduling users in  $\Pi$  in ascending order of  $l_i$  of the requested models maximizes  $\dot{S}_m$ . Similarly,  $\dot{S}_{m,2} - \dot{S}_{m,1} \geq 0$  always holds if  $l_i \geq l_{i_3}$ . This implies that scheduling users in  $\Pi$  in descending order of  $l_i$  of the requested models maximizes  $\dot{S}_m$ . Since we set  $l_0$  to 0, scheduling users in ascending order of  $l_i$  of the requested models preserves the optimality of the user scheduling. This completes the proof of the second statement and concludes the proof of Corollary 1.

## REFERENCES

- [1] T. Zhu, Z. Guo, C. Yao, J. Tan, S. Dou, W. Wang, and Z. Han, "Byzantine-robust federated learning via cosine similarity aggregation," *Comput. Netw.*, vol. 254, p. 110730, Dec. 2024.
- [2] Z. Wang, A. E. Kalør, Y. Zhou, P. Popovski, and K. Huang, "Ultra-low-latency edge inference for distributed sensing," *arXiv preprint arXiv:2407.13360*, 2024.
- [3] Z. Fang, S. Hu, J. Wang, Y. Deng, X. Chen, and Y. Fang, "Prioritized information bottleneck theoretic framework with distributed online learning for edge video analytics," *IEEE/ACM Trans. Netw.*, pp. 1–17, early access 2025.
- [4] J. Huang, K. Yuan, C. Huang, and K. Huang, "D<sup>2</sup>-jscc: Digital deep joint source-channel coding for semantic communications," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 4, pp. 1246–1261, Apr. 2025.
- [5] S. Lyu, Z. Lin, G. Qu, X. Chen, X. Huang, and P. Li, "Optimal resource allocation for U-shaped parallel split learning," in *Proc. IEEE Glob. Commun. Conf. Workshops (GC Wkshps)*, Kuala Lumpur, Malaysia, Dec. 2023, pp. 197–202.
- [6] C. Wu, J. Chen, Z. Wang, R. Liang, and R. Du, "Semantic sleuth: Identifying ponzi contracts via large language models," in *Proc. 39th IEEE/ACM Int. Conf. Autom. Softw. Eng.*, ser. ASE '24, Oct. 2024, p. 582–593.
- [7] ITU Radiocommunication Sector (ITU-R), "Framework and overall objectives of the future development of IMT for 2030 and beyond," International Telecommunication Union (ITU), Recommendation M.2160-0, Nov. 2023, recommendation ITU-R M.2160-0 (11/2023).
- [8] C. Wu, J. Chen, K. He, Z. Zhao, R. Du, and C. Zhang, "EchoHand: High accuracy and presentation attack resistant hand authentication on commodity mobile devices," in *Proc. 2022 ACM SIGSAC Conf. Comput. Commun. Secur.*, ser. CCS '22, Nov. 2022, p. 2931–2945.
- [9] S. Hu, Z. Fang, Y. Deng, X. Chen, Y. Fang, and S. Kwong, "Toward full-scene domain generalization in multi-agent collaborative bird's eye view segmentation for connected and autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 2, pp. 1783–1796, Feb. 2025.
- [10] W. Y. B. Lim, Z. Xiong, D. Niyato, X. Cao, C. Miao, S. Sun, and Q. Yang, "Realizing the metaverse with edge intelligence: A match made in heaven," *IEEE Wireless Commun.*, vol. 30, no. 4, pp. 64–71, Aug. 2022.
- [11] Q. Chen, Z. Wang, X. Chen, J. Wen, D. Zhou, S. Ji, M. Sheng, and K. Huang, "Space-ground fluid AI for 6G edge intelligence," *arXiv preprint arXiv:2411.15845*, 2024.
- [12] W. Dai, H. Nishi, V. Vyatkin, V. Huang, Y. Shi, and X. Guan, "Industrial edge computing: Enabling embedded intelligence," *IEEE Ind. Electron. Mag.*, vol. 13, no. 4, pp. 48–56, Dec. 2019.
- [13] G. Qu, Q. Chen, W. Wei, Z. Lin, X. Chen, and K. Huang, "Mobile edge intelligence for large language models: A contemporary survey," *IEEE Commun. Surveys Tuts.*, pp. 1–42, early access 2025.
- [14] J. Zhang and K. B. Letaief, "Mobile edge intelligence and computing for the internet of vehicles," *Proc. IEEE*, vol. 108, no. 2, pp. 246–261, Feb. 2019.
- [15] S. U. Amin and M. S. Hossain, "Edge intelligence and internet of things in healthcare: A survey," *IEEE Access*, vol. 9, pp. 45–59, Dec. 2020.
- [16] Q. Chen, X. Chen, and K. Huang, "FedMeld: A model-dispersal federated learning framework for space-ground integrated networks," *arXiv preprint arXiv:2412.17231*, 2024.
- [17] W. Wei, J. Wang, J. Du, Z. Fang, Y. Ren, and C. P. Chen, "Differential game-based deep reinforcement learning in underwater target hunting task," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 462–474, Jan. 2023.
- [18] Z. Lin, G. Qu, Q. Chen, X. Chen, Z. Chen, and K. Huang, "Pushing large language models to the 6G edge: Vision, challenges, and opportunities," *arXiv preprint arXiv:2309.16739*, 2023.
- [19] X. Chen, Y. Deng, H. Ding, G. Qu, H. Zhang, P. Li, and Y. Fang, "Vehicle as a service (VaaS): Leverage vehicles to build service networks and capabilities for smart cities," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 3, pp. 2048–2081, 3rd Quart. 2024.
- [20] Q. Chen, Z. Guo, W. Meng, S. Han, C. Li, and T. Q. S. Quek, "A survey on resource management in joint communication and computing-embedded SAGIN," *IEEE Commun. Surveys Tuts.*, early access 2024.
- [21] S. Hu, Z. Fang, Y. Deng, X. Chen, and Y. Fang, "Collaborative perception for connected and autonomous driving: Challenges, possible solutions and opportunities," *arXiv preprint arXiv:2401.01544*, 2024.
- [22] Z. Fang, S. Hu, H. An, Y. Zhang, J. Wang, H. Cao, X. Chen, and Y. Fang, "PACP: Priority-aware collaborative perception for connected and autonomous vehicles," *IEEE Trans. Mobile Comput.*, pp. 15003–15018, Dec. 2024.
- [23] Z. Lin, G. Qu, X. Chen, and K. Huang, "Split learning in 6G edge networks," *IEEE Wireless Commun.*, vol. 31, no. 4, pp. 170–176, Aug. 2024.
- [24] 3GPP, "3rd generation partnership project; Technical specification group services and system aspects; Study on traffic characteristics and performance requirements for AI/ML model transfer in 5GS; (Release 18)," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 22.874, Dec. 2021, version 18.2.0.
- [25] Y. Shi, K. Yang, T. Jiang, J. Zhang, and K. B. Letaief, "Communication-efficient edge AI: Algorithms and systems," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 4, pp. 2167–2191, 4th Quart. 2020.
- [26] J. Shao and J. Zhang, "Communication-computation trade-off in resource-constrained edge inference," *IEEE Commun. Mag.*, vol. 58, no. 12, pp. 20–26, Dec. 2020.
- [27] Z. Liu, Q. Lan, and K. Huang, "Resource allocation for multiuser edge inference with batching and early exiting," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 4, pp. 1186–1200, Apr. 2023.
- [28] Y. Cang, M. Chen, and K. Huang, "Joint batching and scheduling for high-throughput multiuser edge AI with asynchronous task arrivals," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 13 782–13 795, Oct. 2024.
- [29] W. Shi, S. Zhou, Z. Niu, M. Jiang, and L. Geng, "Multiuser co-inference with batch processing capable edge server," *IEEE Trans. Wireless Commun.*, vol. 22, no. 1, pp. 286–300, Jan. 2022.
- [30] X. Han, Z. Cai, Y. Zhang, C. Fan, J. Liu, R. Ma, and R. Buyya, "Hermes: Memory-efficient pipeline inference for large models on edge devices," *arXiv preprint arXiv:2409.04249*, 2024.
- [31] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778.
- [32] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," Apr. 2009.
- [33] TensorFlow, "Transfer learning & fine-tuning," 2023. [Online]. Available: [https://www.tensorflow.org/guide/keras/transfer\\_learning#introduction](https://www.tensorflow.org/guide/keras/transfer_learning#introduction)
- [34] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He, "A comprehensive survey on transfer learning," *Proc. IEEE*, vol. 109, no. 1, pp. 43–76, Jan. 2020.
- [35] G. Qu, Z. Lin, Q. Chen, J. Li, F. Liu, X. Chen, and K. Huang, "TrimCaching: Parameter-sharing edge caching for AI model downloading," *arXiv preprint arXiv:2404.14204*, 2024.
- [36] Y. Guo, H. Shi, A. Kumar, K. Grauman, T. Rosing, and R. Feris, "SpotTune: Transfer learning through adaptive fine-tuning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, Jun. 2019.
- [37] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Montreal, Canada, Dec. 2014, pp. 3320–3328.
- [38] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Apr. 2022, pp. 1–13.
- [39] G. Qu, Z. Lin, F. Liu, X. Chen, and K. Huang, "TrimCaching: Parameter-sharing AI model caching in wireless edge networks," in *Proc. IEEE Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jersey City, NJ, USA, Jul. 2024, pp. 36–46.

- [40] A. Padmanabhan, N. Agarwal, A. Iyer, G. Ananthanarayanan, Y. Shu, N. Karianakis, G. H. Xu, and R. Netravali, "Gemel: Model merging for memory-efficient, real-time video analytics at the edge," in *Proc. USENIX Symp. Netw. Syst. Des. Implement. (NSDI)*, Boston, MA, USA, Apr. 2023, pp. 973–994.
- [41] Y. D. Kwon, J. Chauhan, and C. Mascolo, "YONO: Modeling multiple heterogeneous neural networks on microcontrollers," in *Proc. ACM/IEEE Int. Conf. Inf. Process. Sensor Netw. (IPSN)*, Milano, Italy, May 2022, pp. 285–297.
- [42] X. Zhang, J. Liu, Z. Xiong, Y. Huang, G. Xie, and R. Zhang, "Edge intelligence optimization for large language model inference with batching and quantization," *arXiv preprint arXiv:2405.07140*, 2024.
- [43] Y. Ma, K. Gao, and C. Zhang, "Efficient latency optimization in batch processing server systems for multi-user co-inference," in *Proc. IEEE/CIC Int. Conf. Commun. China Workshops (ICCC Workshops)*, Hangzhou, China, Aug. 2024, pp. 475–480.
- [44] Q. Yang, X. Luo, P. Li, T. Miyazaki, and X. Wang, "Computation offloading for fast CNN inference in edge computing," in *Proc. Conf. Res. Adapt. Conver. Syst.*, Chongqing China, Sep. 2019, pp. 101–106.
- [45] Y. She, T. Shi, J. Wang, and B. Liu, "Dynamic batching and early-exiting for accurate and timely edge inference," in *Proc. IEEE Veh. Technol. Conf. (VTC2024-Spring)*, Singapore, Jun. 2024, pp. 1–6.
- [46] M. Yao, L. Chen, J. Zhang, J. Huang, and J. Wu, "Loading cost-aware model caching and request routing for cooperative edge inference," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Seoul, South Korea, May 2022, pp. 2327–2332.
- [47] S. S. Ogden, G. R. Gilman, R. J. Walls, and T. Guo, "Many models at the edge: Scaling deep inference via model-level caching," in *Proc. IEEE Int. Conf. Auton. Comput. Self-Organizing Syst. (ACSOS)*, Washington, DC, USA, Sep. 2021, pp. 51–60.
- [48] NVIDIA, "Nvidia AMPERE GA102 GPU architecture," 2020. [Online]. Available: <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>
- [49] R. Love, *Linux Kernel Development (3rd Edition)*, 3rd ed., ser. Developer's Library. Addison-Wesley, 2010.
- [50] F. Xu, J. Xu, J. Chen, L. Chen, R. Shang, Z. Zhou, and F. Liu, "iGniter: Interference-aware GPU resource provisioning for predictable DNN inference in the cloud," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 3, pp. 812–827, Mar. 2023.
- [51] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Weinberger, "Multi-scale dense networks for resource efficient image classification," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Vancouver, Canada, Apr. 2018, pp. 1–18.
- [52] H. Wang, T. Li, M. Zhang, Q. Li, H. Cui, Y. Jiang, and Z. Yuan, "Joint configuration optimization and GPU allocation for multi-tenant real-time video analytics on resource-constrained edge," *IEEE Trans. Mobile Comput.*, early access 2024.
- [53] J. Wu, L. Wang, Q. Jin, and F. Liu, "Graft: Efficient inference serving for hybrid deep learning with SLO guarantees via DNN re-alignment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 2, pp. 280–296, Feb. 2024.
- [54] A. Allahverdi, C. T. Ng, T. E. Cheng, and M. Y. Kovalyov, "A survey of scheduling problems with setup times or costs," *Eur. J. Oper. Res.*, vol. 187, no. 3, pp. 985–1032, Jun. 2008.
- [55] P. Brucker and M. Y. Kovalyov, "Single machine batch scheduling to minimize the weighted number of late jobs," *Math. Methods Oper. Res.*, vol. 43, pp. 1–8, Feb. 1996.
- [56] J. Howard and S. Ruder, "Universal language model fine-tuning for text classification," in *Proc. 56th Annu. Meet. Assoc. Comput. Linguistics (ACL)*, Melbourne, Australia, Jul. 2018, pp. 328–339.